Microsoft Interview - 30 High-Level Design Questions

Quick Reference Guide: Functional Requirements + Non-Functional Requirements + Estimations + HLD Diagrams

---

Table of Contents

1. URL Shortener (TinyURL)
2. Social Media Feed (Twitter Timeline)
3. File Storage System (Dropbox/OneDrive)
4. E-Commerce Platform (Amazon)
5. Chat Application (WhatsApp/Teams)
6. Video Streaming (YouTube)
7. Recommendation System (Netflix)
8. Photo Sharing (Instagram)
9. End-to-End Encrypted Messaging (WhatsApp)
10. Ride-Sharing Service (Uber)
11. Location Service (Google Maps)
12. Web Crawler (Googlebot)
13. API Rate Limiter
14. Search Autocomplete
15. Notification System
16. Stock Trading Platform
17. Gaming Leaderboard
18. Job Portal (LinkedIn Jobs)
19. Content Delivery Network (CDN)
20. Distributed Cache (Redis/Memcached)
21. Distributed Task Scheduler
22. Online Meeting Platform (Teams/Zoom)
23. Parking Lot System
24. Metrics Monitoring System
25. Proximity Service (Yelp)
26. Distributed Key-Value Store
27. Ticket Booking System
28. API Gateway
29. News Feed System (LinkedIn/Facebook)
30. Search Engine (Google)

---

1. URL Shortener (TinyURL)

Functional Requirements

- Generate short URL from long URL
- Redirect short URL to original URL

- Custom aliases support
- Analytics (click tracking)
- Expiration time for URLs

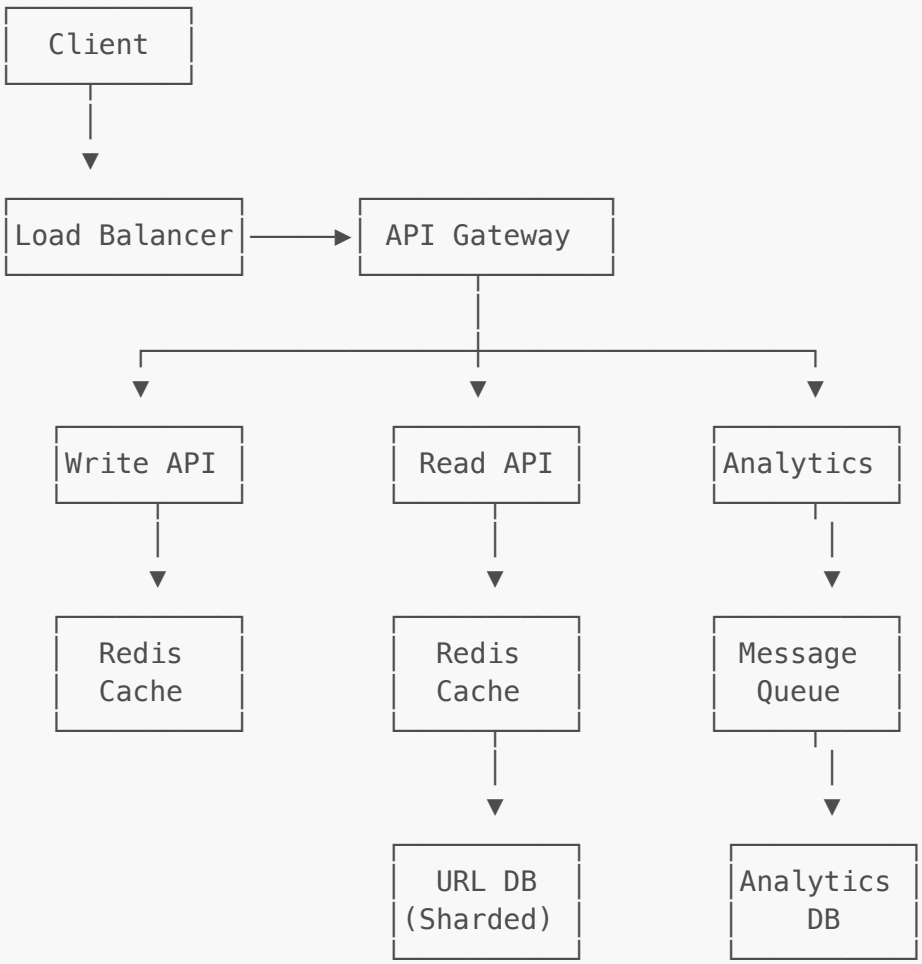
Non-Functional Requirements

- **High Availability**: 99.99%
- **Low Latency**: <100ms
- **Scalability**: 1M URLs/day
- **Durable Storage**
- **No Collision** in short URLs

Back-of-Envelope Estimation

```
DAU: 100M users
Write: 1M new URLs/day = 12 URLs/sec
Read: 100M redirects/day = 1,150 reads/sec
Read:Write Ratio = 100:1
Storage: 1M URLs/day × 365 × 5 years × 500 bytes = 900 GB
URL Length: 7 characters (62^7 = 3.5 trillion combinations)
```

High-Level Design



Key Components

- **Hash Function**: MD5/SHA256 → Base62 encoding
- **Cache**: Redis for hot URLs
- **Database**: Cassandra/DynamoDB (NoSQL)
- **Analytics**: Kafka + Spark

---

2. Social Media Feed (Twitter Timeline)

Functional Requirements

- Post tweets (text, images, videos)
- Follow/unfollow users
- View home timeline (tweets from followed users)
- Like, retweet, reply
- Real-time updates

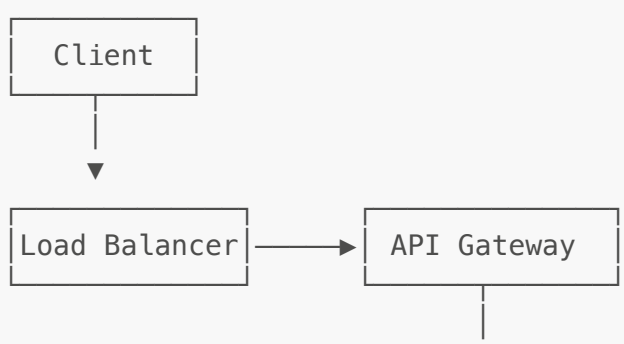
Non-Functional Requirements

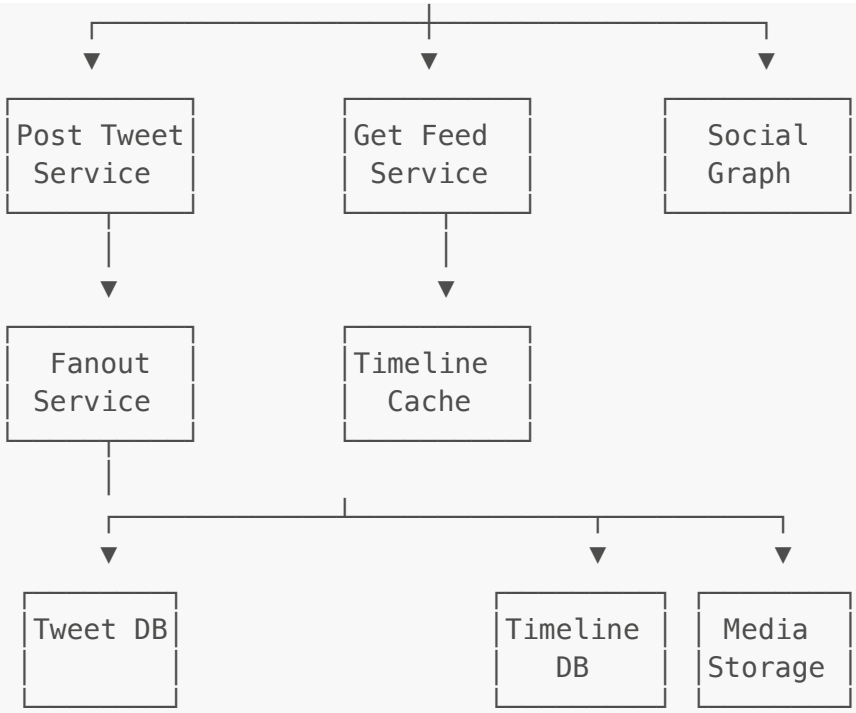
- **High Availability**
- **Low Latency**: <200ms for timeline
- **Eventually Consistent**
- **Handle**: 500M DAU
- **Scale**: Billions of tweets

Back-of-Envelope Estimation

```
DAU: 500M users
Tweets: 200M tweets/day = 2,300 tweets/sec
Timeline Reads: 500M users × 10 reads/day = 58K reads/sec
Avg Followers: 200
Fan-out: 2,300 × 200 = 460K writes/sec to timelines
Storage: 200M tweets × 365 × 280 bytes = 20 TB/year
```

High-Level Design





Key Components

- **Fan-out on Write**: For active users
- **Fan-out on Read**: For celebrities
- **Cache**: Redis for timeline cache
- **Database**: Cassandra for tweet storage
- **CDN**: For media files

---

3. File Storage System (Dropbox/OneDrive)

Functional Requirements

- Upload/download files
- Folder management
- File sharing & permissions
- File versioning
- Sync across devices
- Offline access

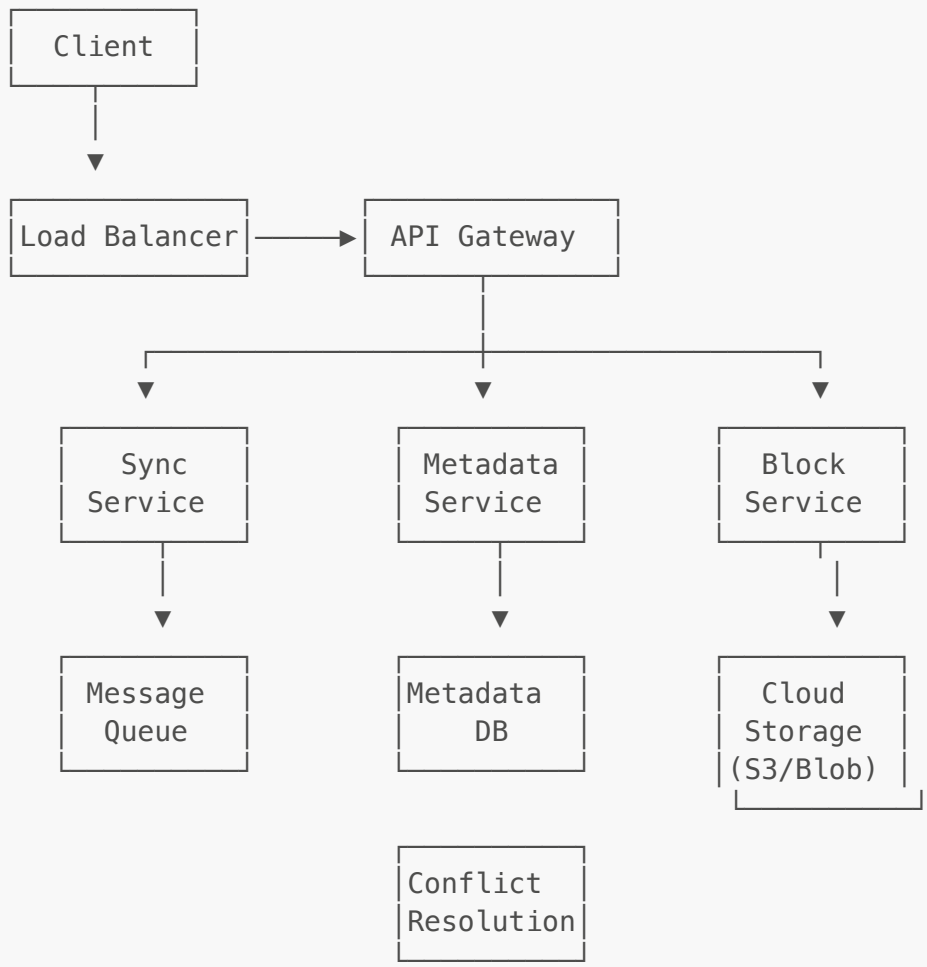
Non-Functional Requirements

- **Availability**: 99.99%
- **Strong Consistency**
- **Handle**: 500M users
- **Support**: Large files (up to 10GB)
- **Efficient Bandwidth** usage

Back-of-Envelope Estimation

```
DAU: 100M users
Total Users: 500M
Avg Storage per User: 10GB
Total Storage: 5 PB
Upload: 10M files/day = 115 files/sec
Download: 50M files/day = 580 files/sec
Avg File Size: 1MB
Bandwidth: Upload 115 MB/sec, Download 580 MB/sec
```

High-Level Design



Key Components

- **Chunking**: Break files into 4MB blocks
- **Deduplication**: SHA-256 hash
- **Metadata DB**: MySQL/PostgreSQL
- **Block Storage**: S3/Azure Blob
- **Notification**: WebSocket for real-time sync

---

4. E-Commerce Platform (Amazon)

Functional Requirements

- Product catalog & search
- Shopping cart
- Order placement
- Payment processing
- Inventory management
- Order tracking
- Reviews & ratings

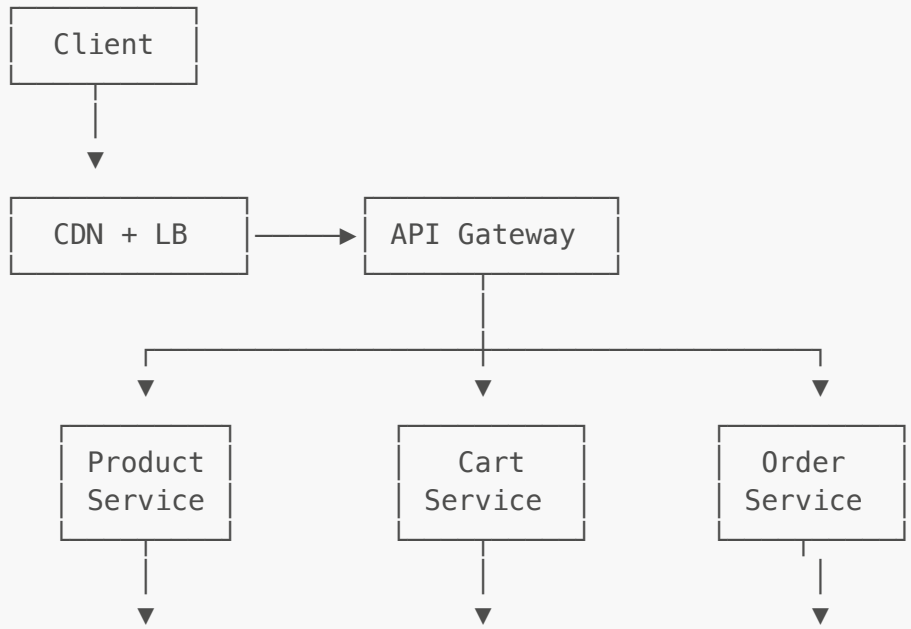
Non-Functional Requirements

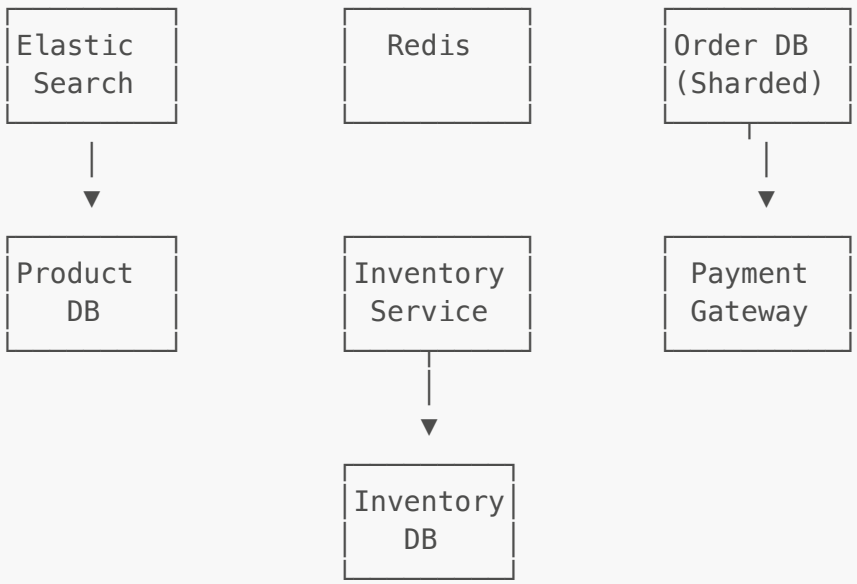
- **High Availability**: 99.99%
- **ACID Transactions** for orders
- **Low Latency Search**: <100ms
- **Handle**: 10M DAU
- **Peak Traffic**: 10x normal

Back-of-Envelope Estimation

```
DAU: 10M users
Products: 100M SKUs
Orders: 1M orders/day = 12 orders/sec
Peak: 120 orders/sec
Search QPS: 100K queries/sec
Cart Operations: 500K/sec
Avg Order Value: $50
Daily Revenue: $50M
```

High-Level Design





Key Components

- **Search**: Elasticsearch
- **Cache**: Redis for products & cart
- **Database**: PostgreSQL with sharding
- **Payment**: Stripe/PayPal integration
- **Inventory**: 2PC or Saga pattern

---

5. Chat Application (WhatsApp/Teams)

Functional Requirements

- 1-to-1 messaging
- Group chats (up to 256 members)
- Online/offline status
- Message delivery status (sent/delivered/read)
- Media sharing (images/videos/files)
- Push notifications

Non-Functional Requirements

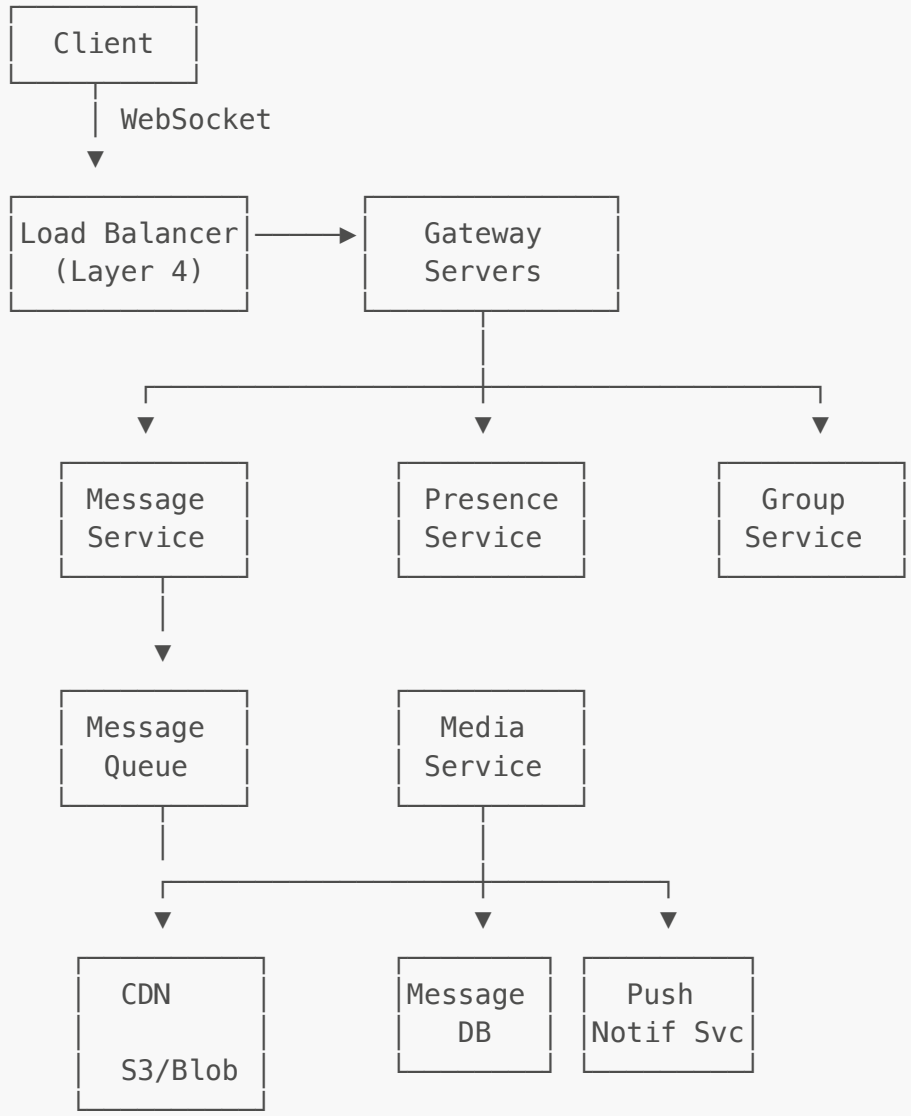
- **High Availability**
- **Low Latency**: <100ms
- **Real-time Delivery**
- **Handle**: 2B users
- **End-to-End Encryption**

Back-of-Envelope Estimation

```
DAU: 500M users
Messages: 100B messages/day = 1.15M msgs/sec
```

```
Group Messages: 20% = 230K msgs/sec
Average Message Size: 100 bytes
Storage: 100B × 100 bytes = 10 TB/day
Media: 20% of messages = 20B files/day
Concurrent Connections: 500M WebSocket connections
```

High-Level Design



Key Components

- **WebSocket**: For real-time communication
- **Message Queue**: Kafka
- **Database**: Cassandra (wide-column)
- **Presence**: Redis with TTL
- **Media**: S3 + CDN

---

6. Video Streaming (YouTube)

Functional Requirements

- Upload videos
- Stream videos (adaptive bitrate)
- Search videos
- Like/comment/subscribe
- Recommendations
- View count & analytics

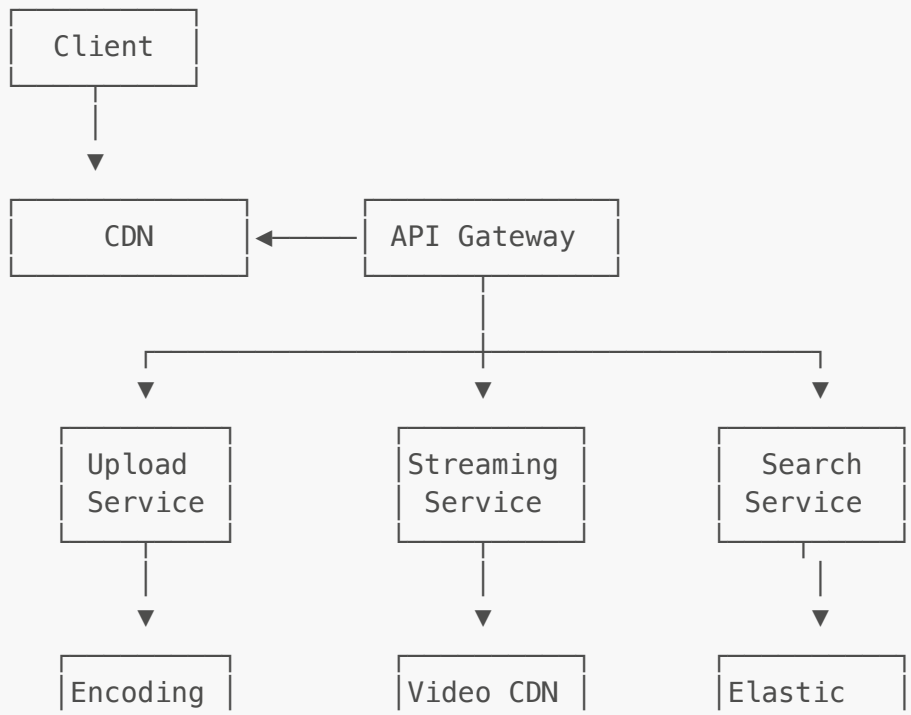
Non-Functional Requirements

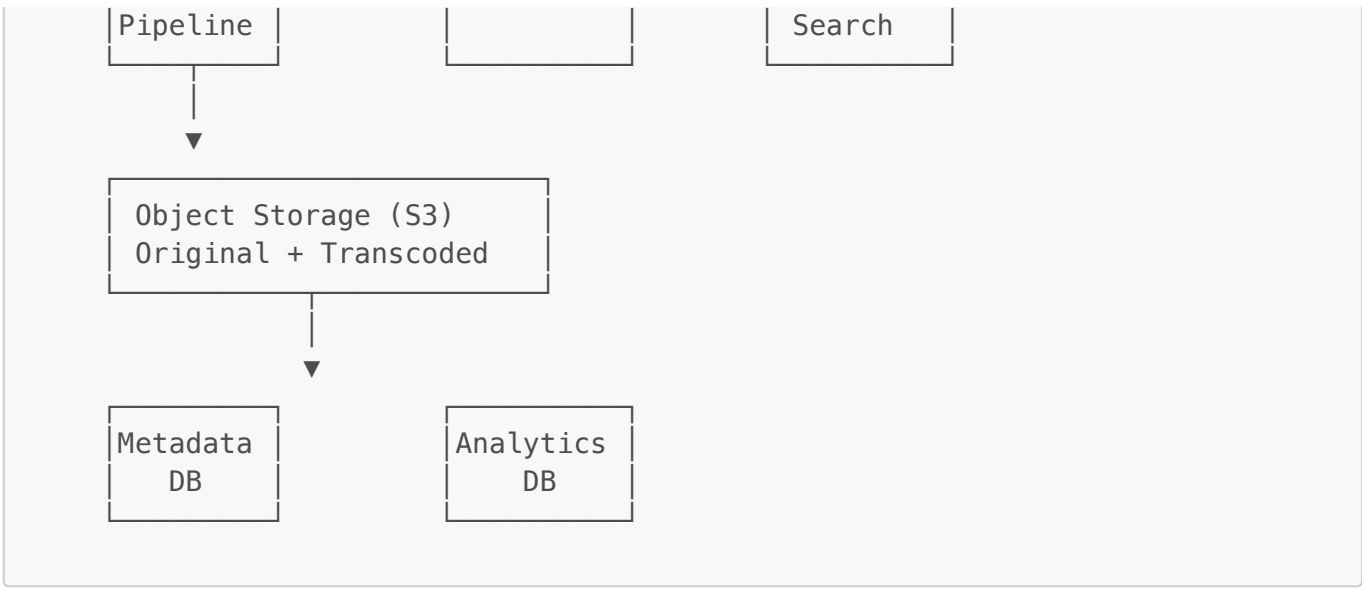
- **High Availability**
- **Low Latency** streaming
- **Handle**: 2B users
- **Support**: 4K videos
- **CDN** for global distribution

Back-of-Envelope Estimation

```
DAU: 500M users
Video Uploads: 500K videos/day = 6 videos/sec
Video Views: 5B views/day = 58K views/sec
Avg Video Size: 100MB (before encoding)
Storage: 500K × 100MB = 50 TB/day (raw)
After Compression (5 formats): 250 TB/day
Bandwidth: 58K × 3Mbps = 174 Gbps
```

High-Level Design





Key Components

- **Encoding**: FFmpeg (multiple resolutions)
- **Formats**: 144p, 360p, 720p, 1080p, 4K
- **Protocol**: HLS/DASH
- **CDN**: CloudFront/Akamai
- **Recommendation**: ML models

---

7. Recommendation System (Netflix)

Functional Requirements

- Video streaming
- Personalized recommendations
- Search & browse
- User profiles
- Continue watching
- Download for offline

Non-Functional Requirements

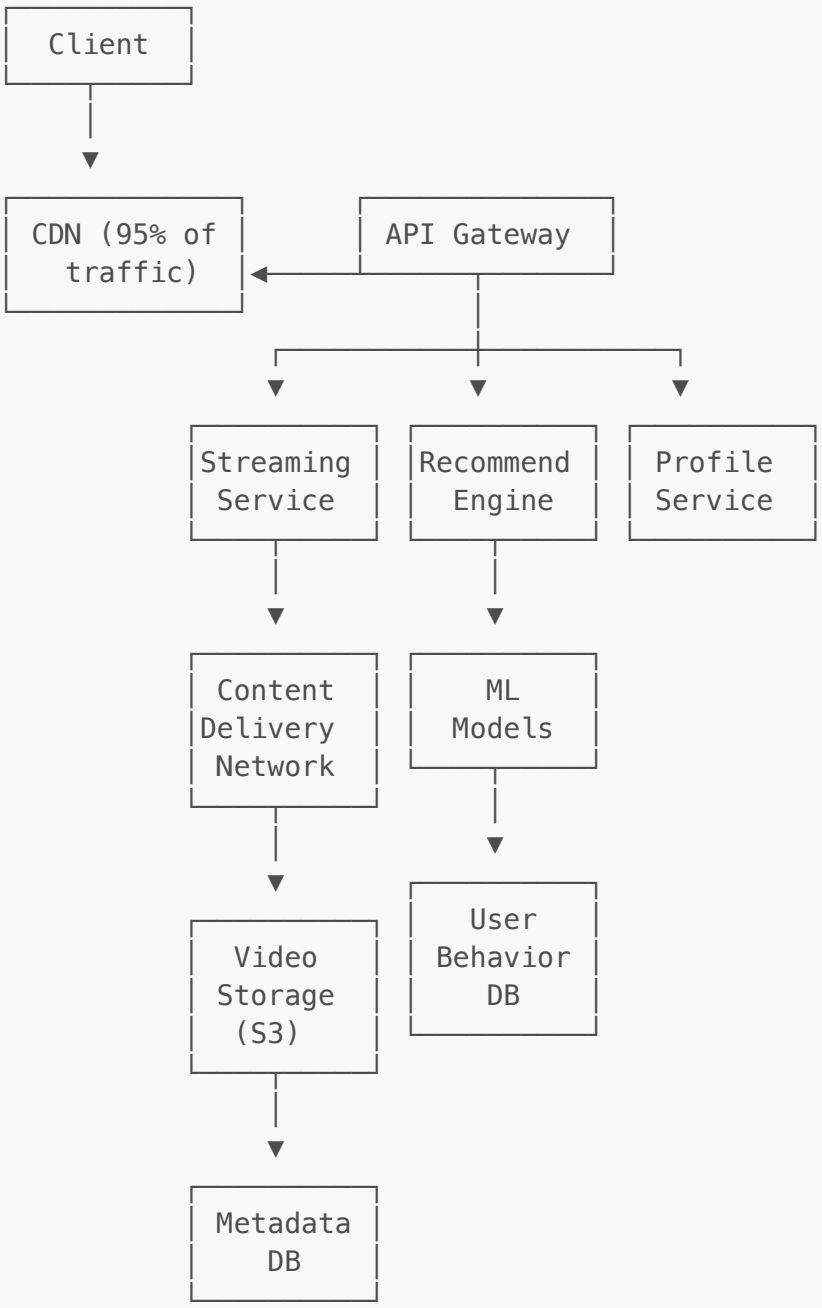
- **Availability**: 99.99%
- **Low Latency**: <2s to start
- **Handle**: 200M users
- **High-Quality** streaming
- **Global Distribution**

Back-of-Envelope Estimation

```
Total Users: 200M
DAU: 100M users
Concurrent Streams: 10M
Avg Bandwidth per Stream: 5 Mbps
```

```
Total Bandwidth: 50 Tbps
Content Library: 10K titles
Avg Video Size: 5GB (multiple formats)
Storage: 50 TB (with redundancy: 150 TB)
```

High-Level Design



Key Components

- **CDN**: Open Connect (Netflix's own)
- **Recommendation**: Collaborative filtering
- **Encoding**: Per-title encoding
- **Caching**: Popular content at edge

- **Database**: Cassandra for user data

---

8. Photo Sharing (Instagram)

Functional Requirements

- Upload photos/videos
- Apply filters
- Follow/unfollow users
- News feed
- Stories (24-hour expiry)
- Direct messaging
- Like, comment, share

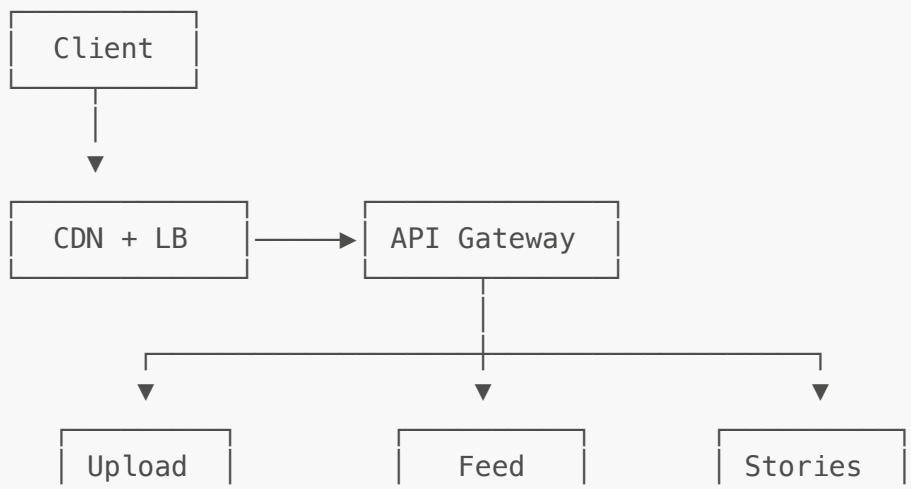
Non-Functional Requirements

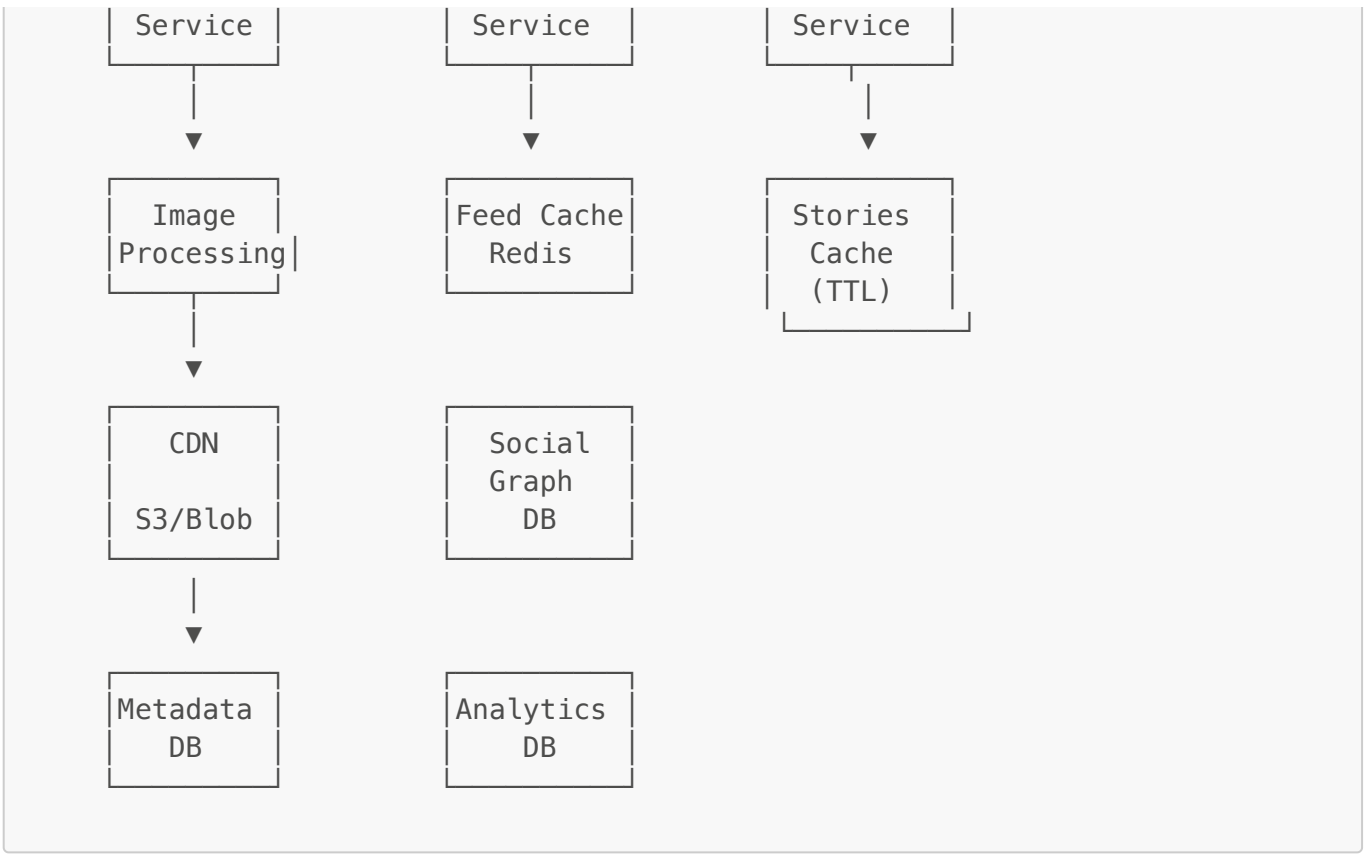
- **High Availability**
- **Low Latency Feed**: <200ms
- **Handle**: 2B users
- **Real-time Updates**
- **Efficient Storage**

Back-of-Envelope Estimation

```
DAU: 500M users
Photos Uploaded: 100M/day = 1,150/sec
Feed Views: 500M × 10 = 5B/day = 58K/sec
Avg Photo Size: 2MB (original), 200KB (compressed)
Storage: 100M × 200KB = 20 TB/day
Stories: 50M/day × 5MB = 250 TB/day (deleted after 24h)
Read:Write Ratio = 50:1
```

High-Level Design





Key Components

- **Image Processing**: Resize, filters
- **Feed**: Fan-out on write + read hybrid
- **Stories**: Redis with TTL (24 hours)
- **Storage**: S3 + CloudFront CDN
- **Graph DB**: For social connections

---

9. End-to-End Encrypted Messaging (WhatsApp)

Functional Requirements

- 1-to-1 and group messaging
- End-to-end encryption
- Media sharing
- Voice/video calls
- Status updates
- Message backup

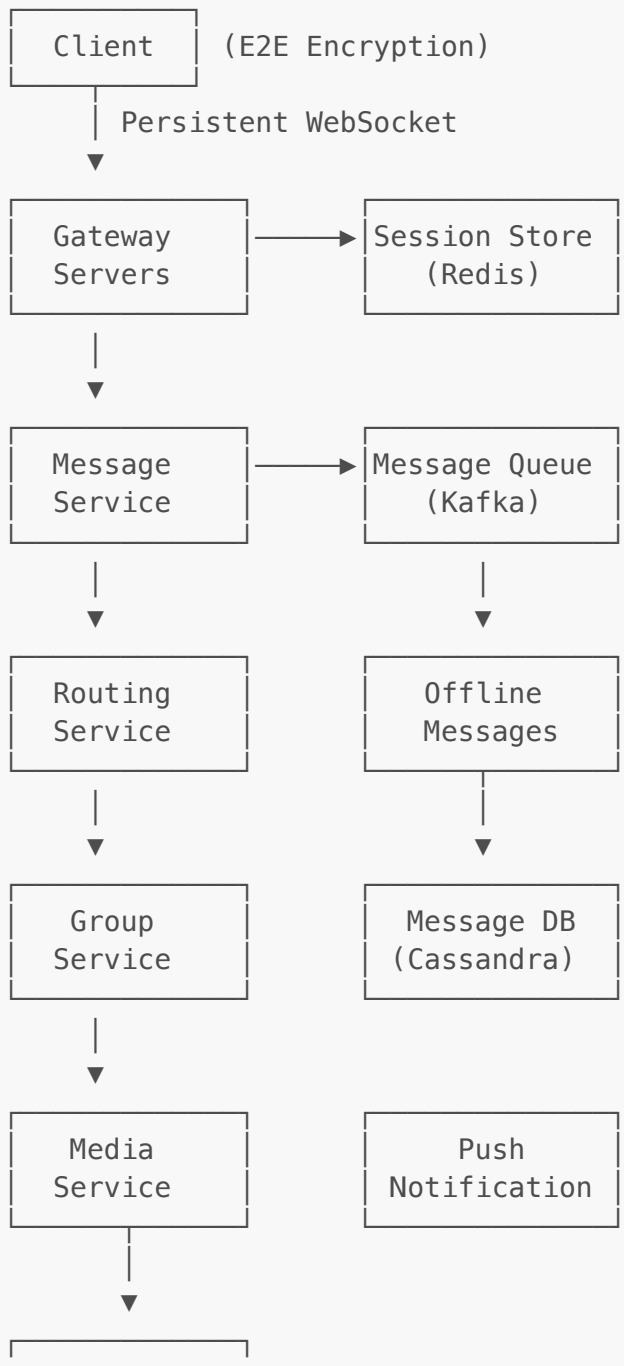
Non-Functional Requirements

- **Availability**: 99.999%
- **Latency**: <100ms
- **Handle**: 2B users
- **Strong Security**
- **Low Bandwidth** usage

Back-of-Envelope Estimation

```
DAU: 1B users
Messages: 100B/day = 1.15M/sec
Group Messages: 25% = 290K/sec
Concurrent Connections: 1B
Avg Message: 100 bytes
Daily Data: 10 TB
Media Messages: 30% = 30B/day
Calls: 10M concurrent
```

High-Level Design



```
│   S3/CDN    │
└──────────────┘
```

Key Components

- **Encryption**: Signal Protocol
- **WebSocket**: Persistent connections
- **Message Queue**: Kafka for async processing
- **Database**: Cassandra for messages
- **Media**: S3 with encryption

---

10. Ride-Sharing Service (Uber)

Functional Requirements

- Request ride
- Match driver with rider
- Real-time location tracking
- ETA calculation
- Fare calculation
- Payment processing
- Driver ratings

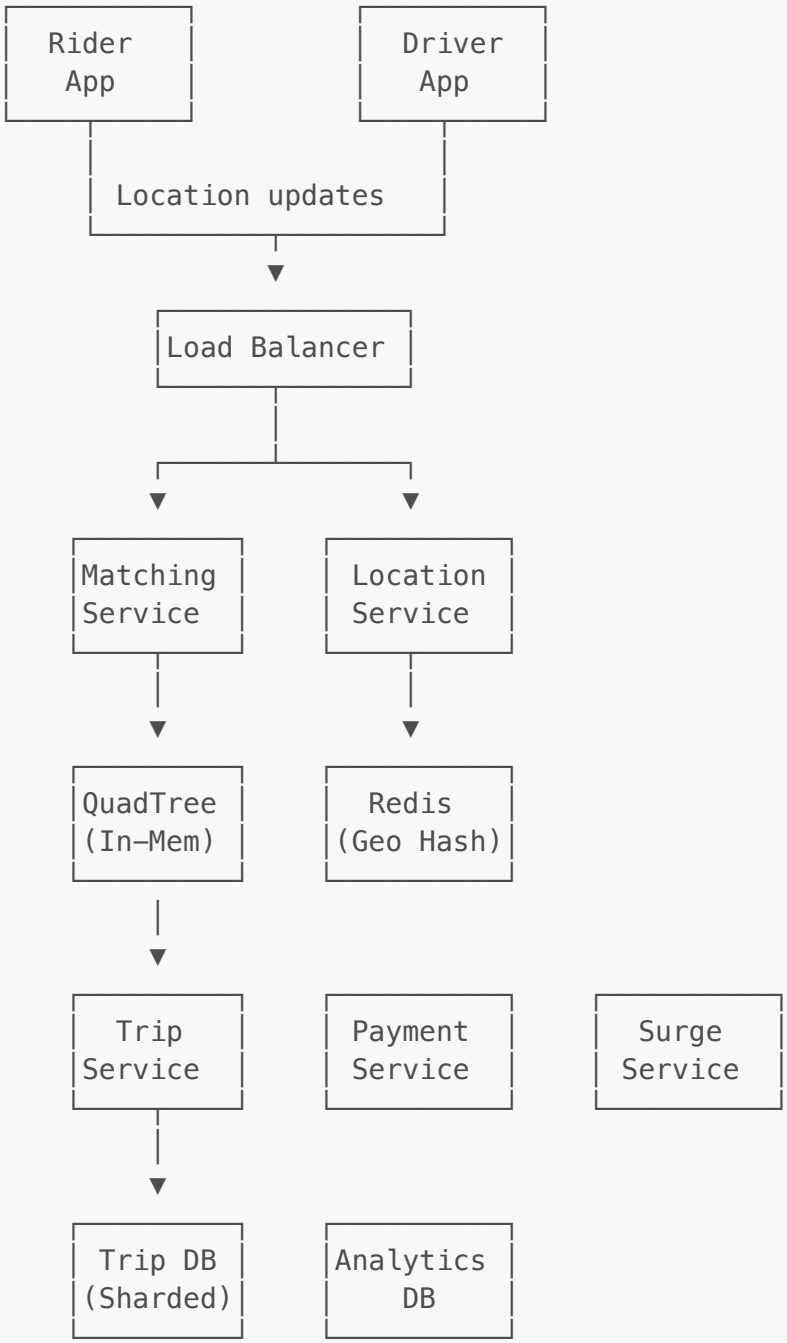
Non-Functional Requirements

- **High Availability**
- **Low Latency Matching**: <5 sec
- **Handle**: 50M DAU
- **Accurate Location** tracking
- **Surge Pricing** support

Back-of-Envelope Estimation

```
DAU: 50M users (riders + drivers)
Active Drivers: 5M
Rides/day: 20M = 230 rides/sec
Location Updates: 5M × 1 update/4sec = 1.25M/sec
Peak Hours: 3x = 690 rides/sec
Avg Ride Duration: 15 min
Concurrent Rides: 2M
Storage: Location data = 1.25M × 50 bytes = 62.5 MB/sec
```

High-Level Design



Key Components

- **Geospatial**: QuadTree/Geohash
- **Matching**: Distance + availability algorithm
- **Location**: Redis Geo
- **Routing**: Google Maps API
- **Surge**: Supply-demand algorithm

---

11. Location Service (Google Maps)

Functional Requirements

- Display maps

- Search locations
- Turn-by-turn navigation
- Real-time traffic
- ETA calculation
- Route optimization
- Street view

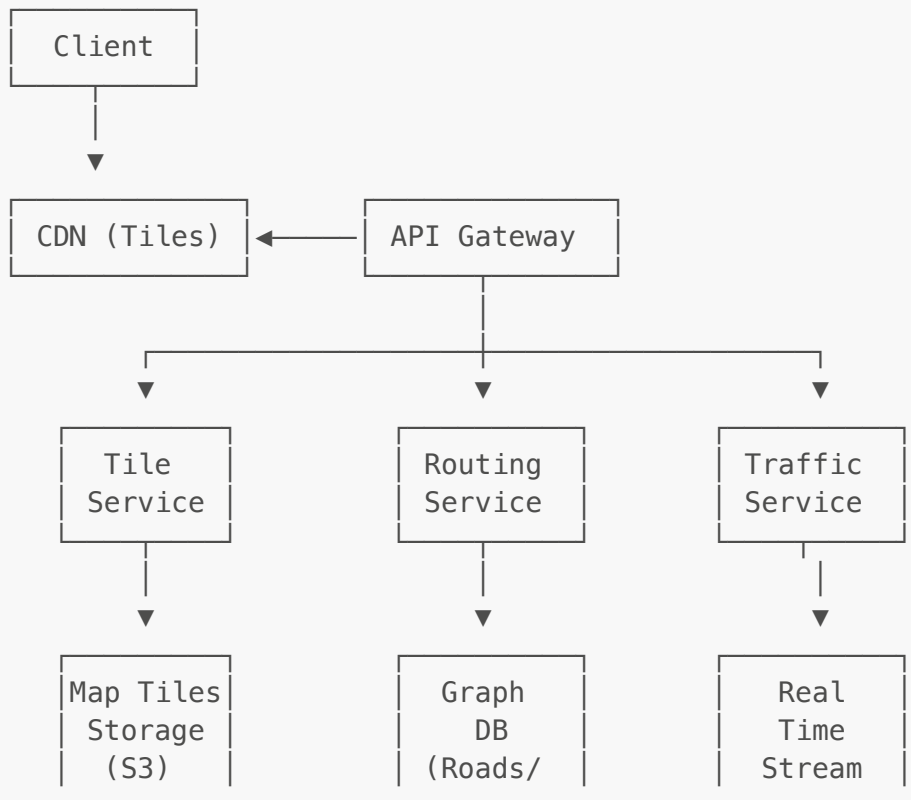
Non-Functional Requirements

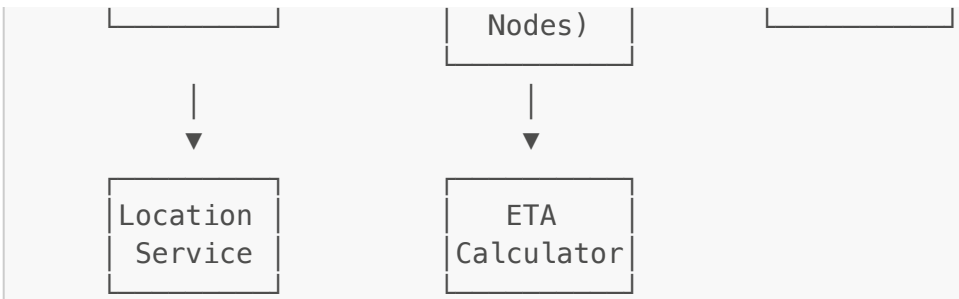
- **Availability**: 99.99%
- **Low Latency**: <100ms
- **Handle**: 1B users
- **Accurate Routing**
- **Real-time Updates**

Back-of-Envelope Estimation

```
DAU: 200M users
Map Tile Requests: 1B/day = 11.5K/sec
Navigation Sessions: 50M/day = 580/sec
Location Updates: 50M × 1/sec = 50M/sec
Tile Size: 256×256 pixels = 50KB
Tile Cache: 100M tiles × 50KB = 5 PB
Traffic Data: 50M updates/sec × 20 bytes = 1 GB/sec
```

High-Level Design





Key Components

- **Map Tiles**: Pre-rendered, cached
- **Routing**: Dijkstra/A* algorithm
- **Graph DB**: Roads as edges
- **Traffic**: Real-time ML models
- **Geohashing**: For location indexing

---

12. Web Crawler (Googlebot)

Functional Requirements

- Crawl web pages
- Extract links
- Store content
- Respect robots.txt
- Politeness policy
- Distributed crawling
- Duplicate detection

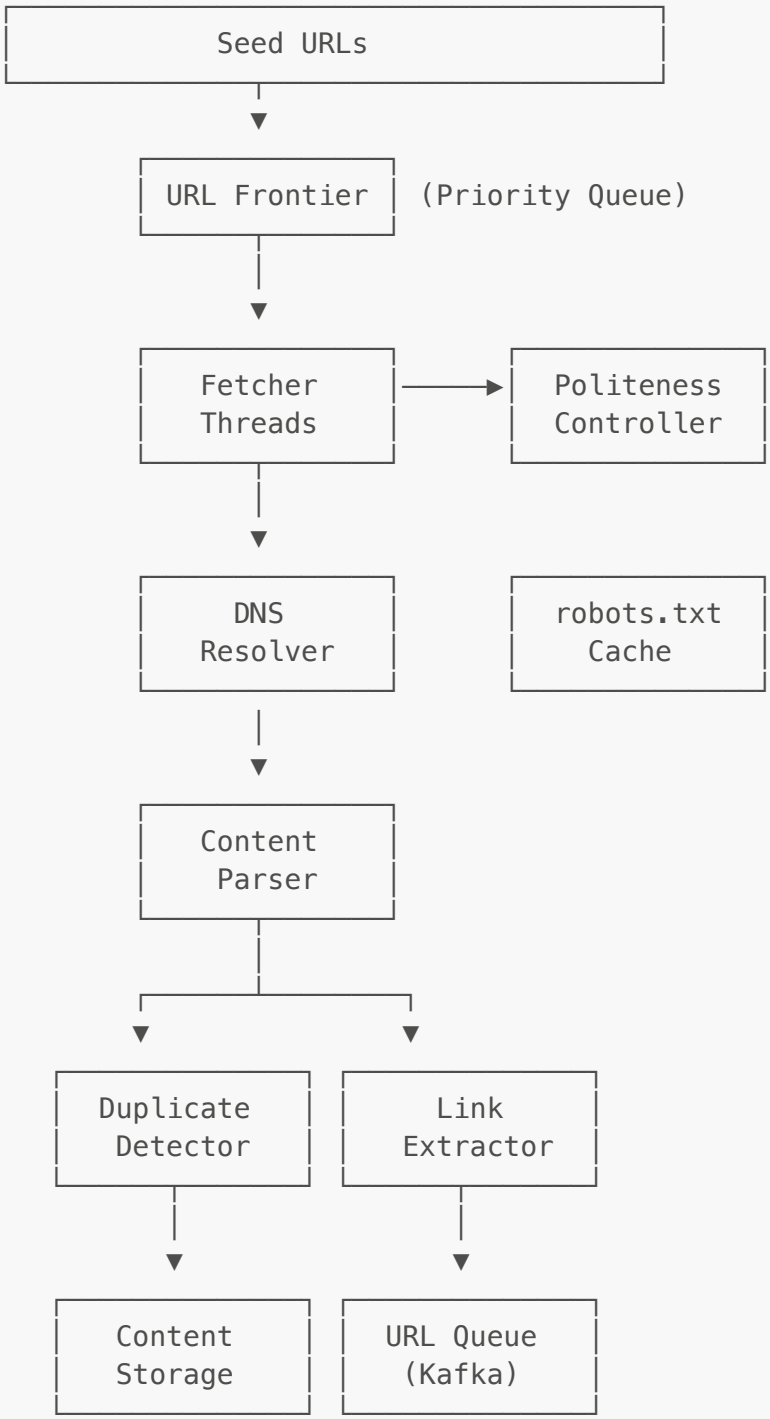
Non-Functional Requirements

- **Scalable**: Billions of pages
- **Fault Tolerant**
- **Efficient Bandwidth** usage
- **Handle Dynamic** content
- **Prioritize Important** pages

Back-of-Envelope Estimation

```
Total Web Pages: 50B
Crawl Frequency: 1 month
Pages/day: 50B/30 = 1.7B/day = 19K/sec
Avg Page Size: 100KB
Daily Data: 1.7B × 100KB = 170 TB/day
Storage (compressed): 50B × 50KB = 2.5 PB
Concurrent Connections: 10K
```

High-Level Design



Key Components

- **URL Frontier**: Priority queue
- **Bloom Filter**: Duplicate detection
- **DNS Cache**: Reduce lookups
- **Politeness**: Delay between requests
- **Storage**: Distributed file system

---

13. API Rate Limiter

Functional Requirements

- Limit requests per user/IP/API key
- Multiple rate limit rules (per sec/min/hour)
- Throttle or reject excess requests
- Return rate limit info in headers
- Configurable limits per tier

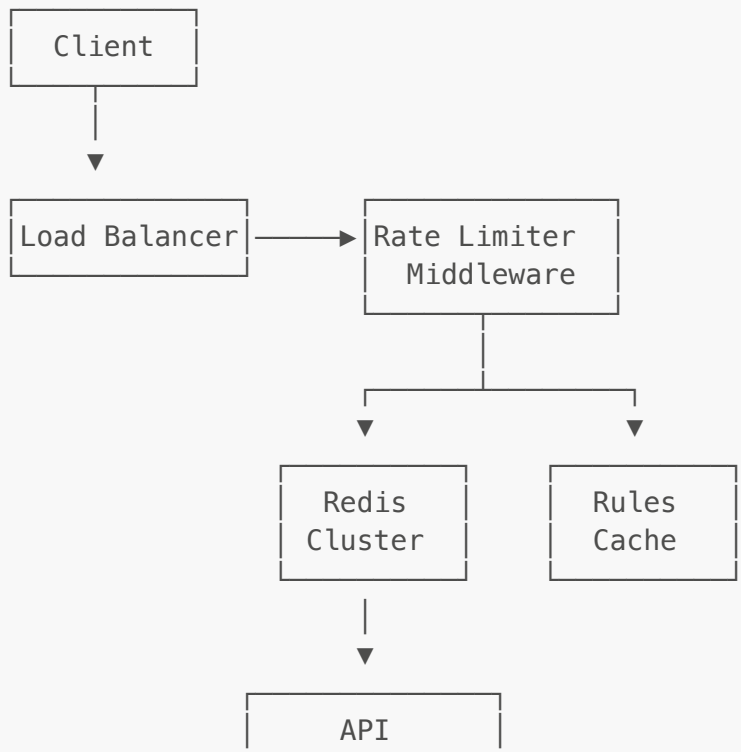
Non-Functional Requirements

- **Low Latency**: <10ms
- **Accurate Counting**
- **Distributed Support**
- **Handle**: 10K requests/sec
- **Fault Tolerant**

Back-of-Envelope Estimation

```
API Requests: 10K/sec
Users: 1M
Rate Limit Check: <10ms
Memory per User: 100 bytes
Total Memory: 100 MB
Redis Operations: 20K/sec (read+write)
```

High-Level Design



```
                │   Service    │
                └──────────────┘

Response Headers:
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 50
X-RateLimit-Reset: 1640000000
```

Algorithms

Token Bucket

- Tokens refilled at rate `r`
- Bucket capacity: `b`
- Request consumes 1 token

Leaky Bucket

- Queue requests
- Process at fixed rate

Fixed Window

- Counter resets every window

Sliding Window Log

- Store timestamp of each request
- Remove old entries

Sliding Window Counter

- Hybrid of fixed window + log

Key Components

- **Redis**: Distributed counter
- **Algorithm**: Token bucket / Sliding window
- **Config**: Per user/tier limits
- **Response**: 429 Too Many Requests

---

14. Search Autocomplete

Functional Requirements

- Suggest queries as user types
- Top 5-10 suggestions
- Support prefix matching

- Personalized suggestions
- Trending queries
- Spell correction

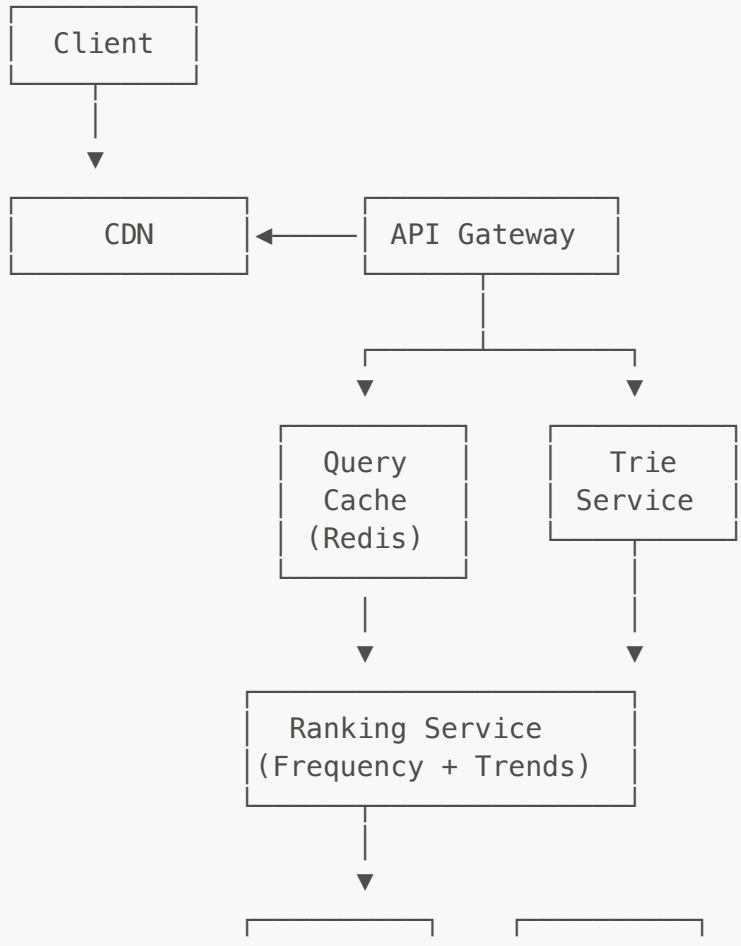
Non-Functional Requirements

- **Low Latency**: <100ms
- **High Availability**
- **Handle**: 100K QPS
- **Real-time Updates**
- **Scalable Storage**

Back-of-Envelope Estimation

```
DAU: 500M users
Queries: 5B/day = 58K QPS
Autocomplete Requests: 5B × 5 chars = 25B/day = 290K QPS
Unique Queries: 100M
Avg Query Length: 20 bytes
Storage: 100M × 20 bytes = 2 GB
Trie Nodes: 500M nodes × 100 bytes = 50 GB
```

High-Level Design



```
                │Analytics │     │  Query   │
                │  Engine  │     │   Log    │
                └──────────┘     └──────────┘
```

Key Components

- **Trie**: Prefix tree in memory
- **Cache**: Redis for popular queries
- **Ranking**: Frequency + recency
- **Personalization**: User history
- **Update**: Async batch updates

---

15. Notification System

Functional Requirements

- Send notifications (email, SMS, push)
- Multiple channels
- Priority levels
- Scheduled notifications
- User preferences
- Retry failed notifications
- Template support

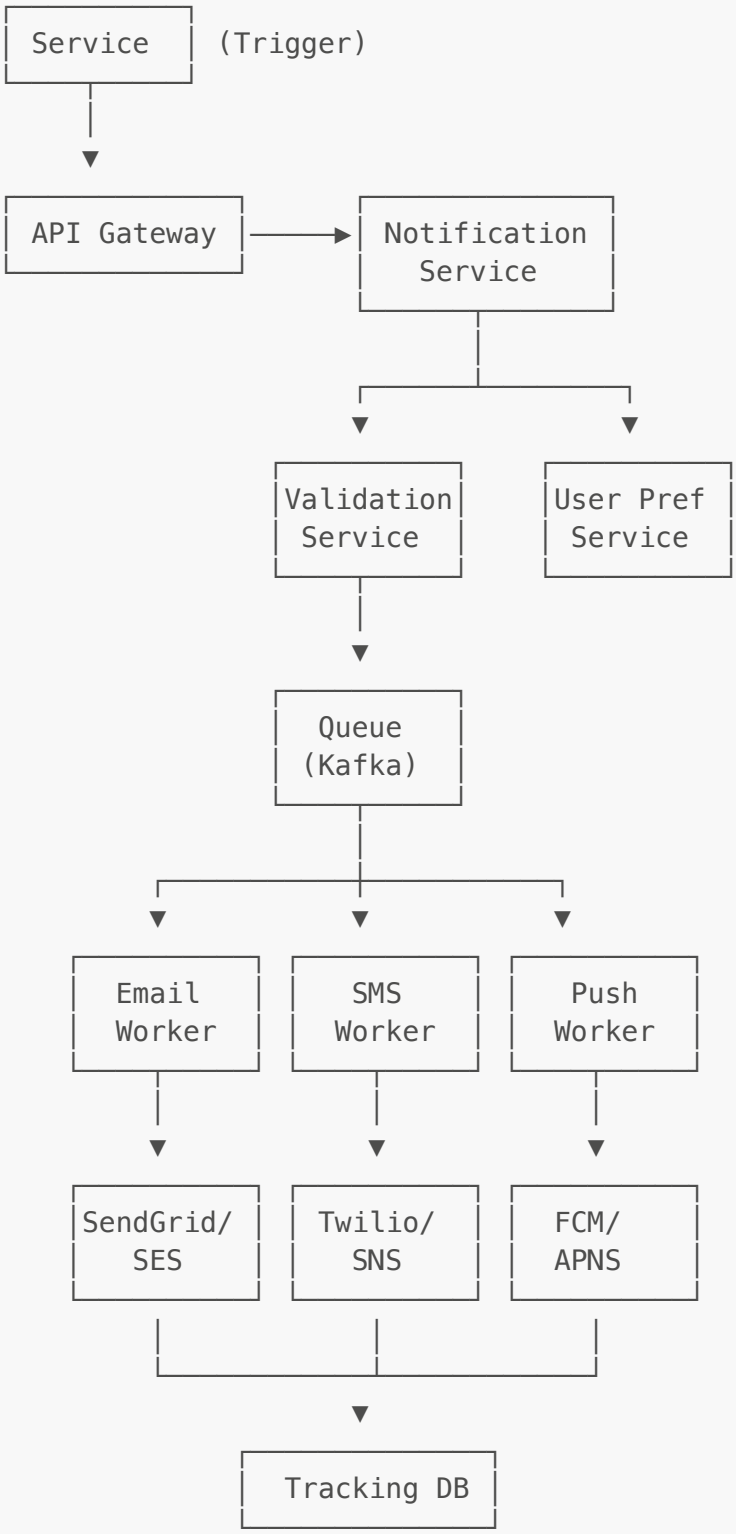
Non-Functional Requirements

- **High Availability**
- **Low Latency**: <5 sec
- **Handle**: 10M notifications/day
- **Reliable Delivery**
- **Rate Limiting** per channel

Back-of-Envelope Estimation

```
Notifications/day: 10M
Peak: 50M/day = 580/sec
Email: 40% = 4M
SMS: 20% = 2M
Push: 40% = 4M
Avg Notification Size: 1KB
Storage: 10M × 1KB = 10 GB/day
Retry Attempts: 3x
```

High-Level Design



Key Components

- **Queue**: Kafka for buffering
- **Workers**: Per channel processing
- **Retry**: Exponential backoff
- **Rate Limiting**: Token bucket
- **Templates**: Stored in DB

---

16. Stock Trading Platform

Functional Requirements

- Place orders (market/limit)
- Order matching
- Real-time stock prices
- Portfolio management
- Trade history
- Market data feed

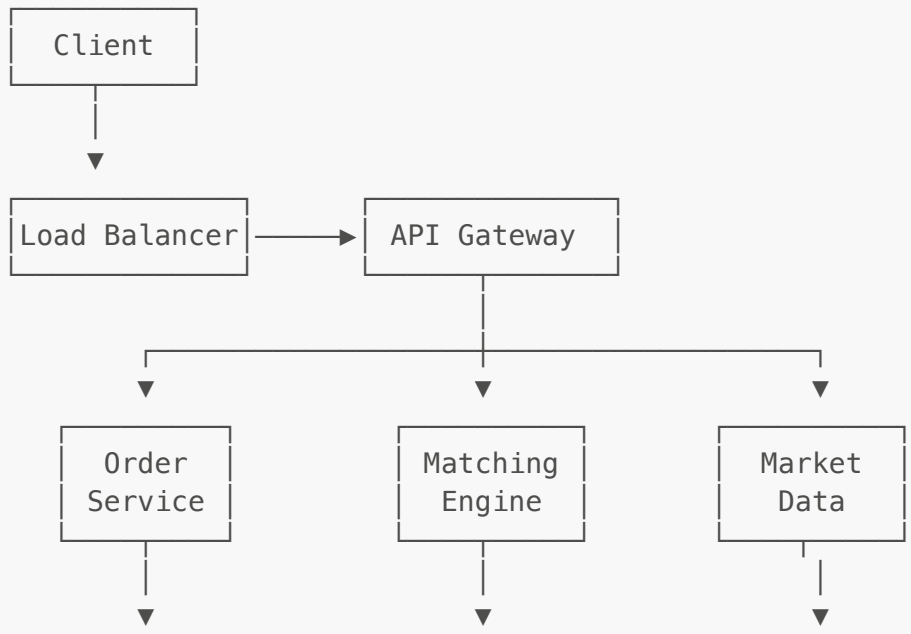
Non-Functional Requirements

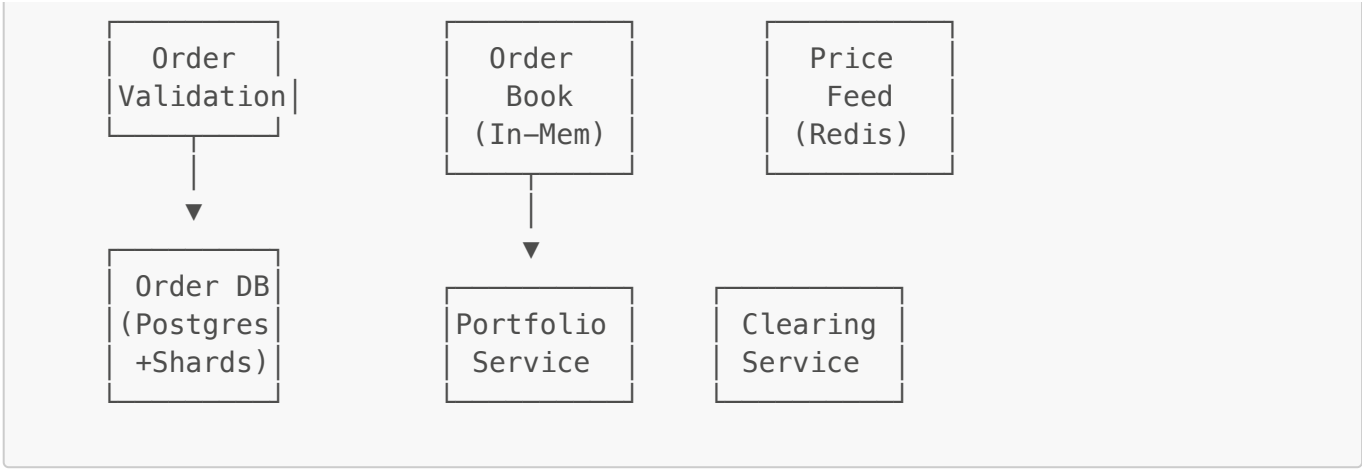
- **Low Latency**: <10ms
- **ACID Transactions**
- **Handle**: 1M trades/day
- **High Availability**
- **Strong Consistency**

Back-of-Envelope Estimation

```
Users: 10M
DAU: 1M traders
Orders/day: 10M = 115 orders/sec
Peak: 1,000 orders/sec
Market Data Updates: 10K stocks × 1 update/sec = 10K/sec
Order Size: 200 bytes
Daily Storage: 10M × 200 bytes = 2 GB
```

High-Level Design





Key Components

- **Matching Engine**: FIFO order book
- **Order Book**: In-memory (price-time priority)
- **Database**: PostgreSQL with ACID
- **Market Data**: Redis pub/sub
- **Low Latency**: C++/Java implementation

---

17. Gaming Leaderboard

Functional Requirements

- Update scores in real-time
- Get top N players
- Get player rank
- Get players around a rank
- Multiple leaderboards (daily/weekly/all-time)
- Handle millions of players

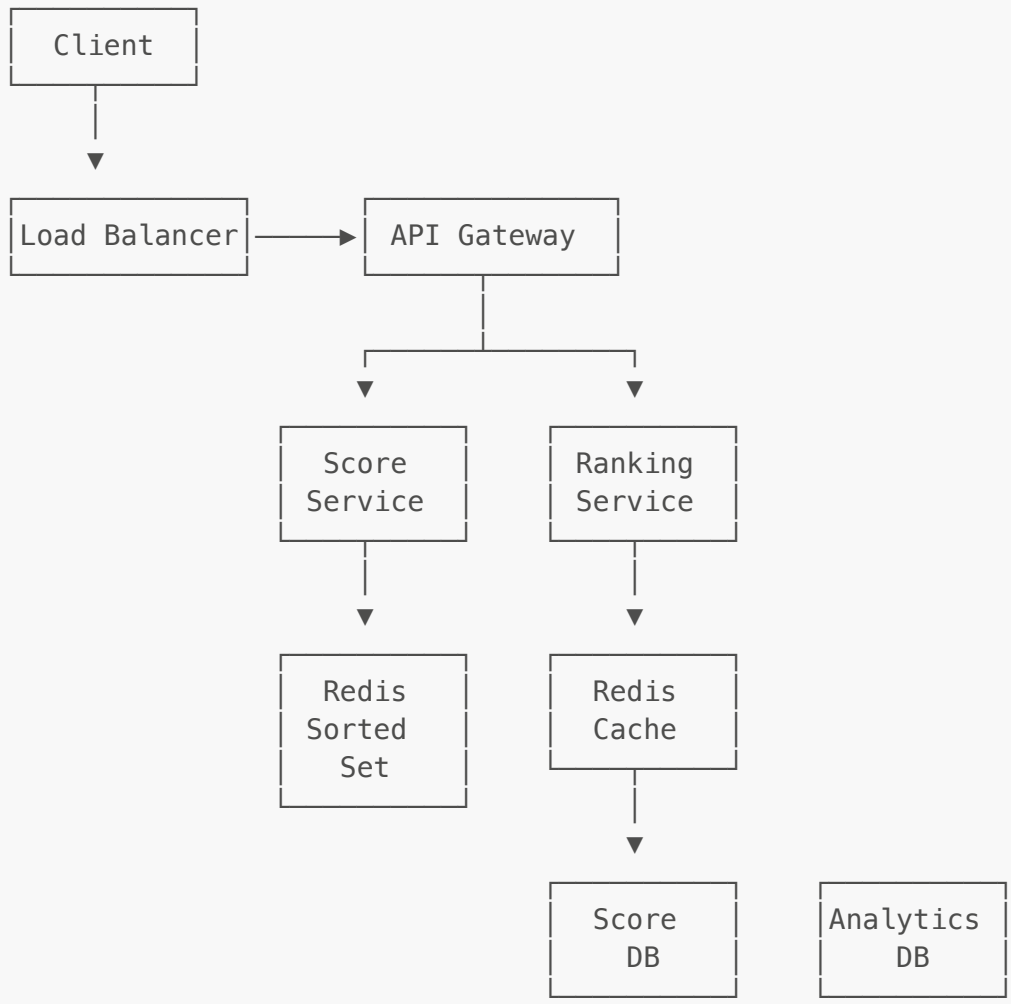
Non-Functional Requirements

- **Low Latency**: <50ms
- **High Availability**
- **Handle**: 10M updates/day
- **Consistent Rankings**
- **Scalable**

Back-of-Envelope Estimation

```
Players: 100M
Active Players/day: 10M
Score Updates: 100M/day = 1,150/sec
Leaderboard Queries: 50M/day = 580/sec
Top 100 Queries: 40M/day
Storage: 100M players × 50 bytes = 5 GB
```

High-Level Design



Redis Sorted Set Operations

```
ZADD leaderboard score player_id
ZREVRANK leaderboard player_id (get rank)
ZREVRANGE leaderboard 0 99 (top 100)
ZREVRANGE leaderboard start end (around rank)
```

Key Components

- **Redis Sorted Set**: O(log n) operations
- **Sharding**: By region/game mode
- **Cache**: Top 100 cached
- **Batch Updates**: Reduce write load
- **Periodic Snapshots**: To DB

---

18. Job Portal (LinkedIn Jobs)

Functional Requirements

- Post jobs
- Search jobs
- Apply for jobs
- Job recommendations
- Candidate matching
- Application tracking
- Company pages

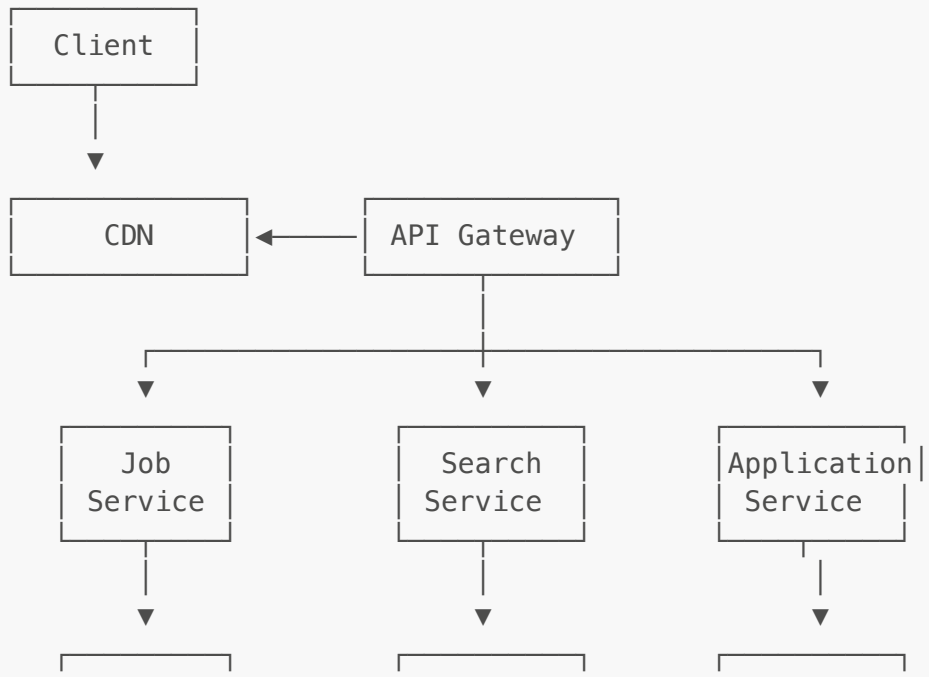
Non-Functional Requirements

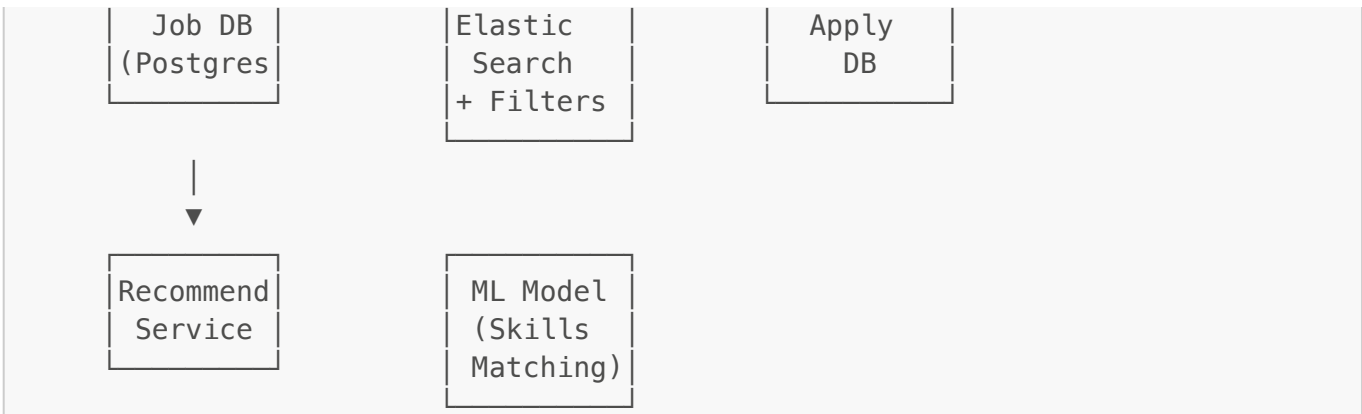
- **High Availability**
- **Low Latency Search**: <200ms
- **Handle**: 100M users
- **Accurate Recommendations**
- **Scalable**

Back-of-Envelope Estimation

```
Users: 100M
Jobs: 10M active
Applications/day: 5M = 58/sec
Search Queries: 50M/day = 580/sec
Job Postings: 100K/day = 1.2/sec
Storage: Jobs 10M × 5KB = 50 GB
User Profiles: 100M × 10KB = 1 TB
```

High-Level Design





Key Components

- **Search**: Elasticsearch with filters
- **Recommendations**: Collaborative filtering
- **Matching**: Skill-based ML models
- **Database**: PostgreSQL for jobs
- **Cache**: Redis for hot jobs

---

19. Content Delivery Network (CDN)

Functional Requirements

- Cache static content
- Serve content from edge
- Cache invalidation
- SSL/TLS support
- Compression
- Geographic routing

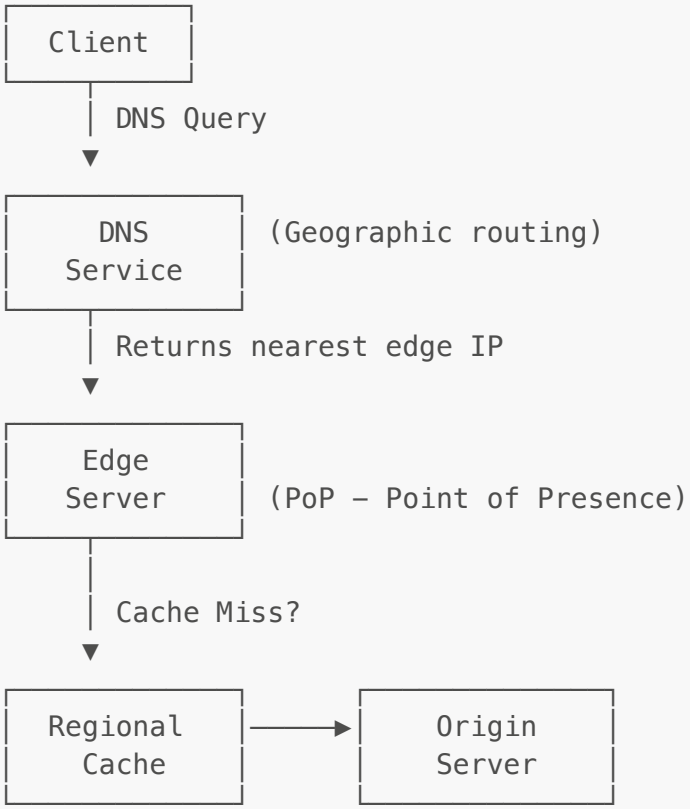
Non-Functional Requirements

- **Low Latency**: <50ms
- **High Availability**: 99.99%
- **Handle**: 100K requests/sec
- **Global Distribution**
- **DDoS Protection**

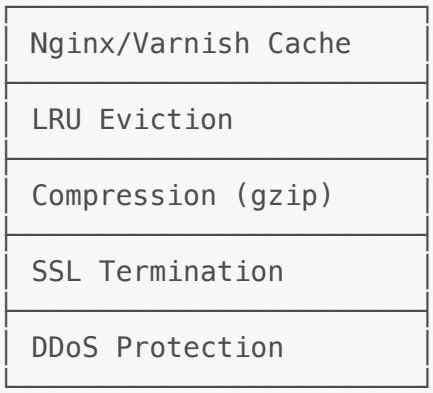
Back-of-Envelope Estimation

```
Requests/day: 10B = 115K/sec
Content Size: Avg 100KB
Cache Hit Ratio: 90%
Origin Requests: 11.5K/sec
Bandwidth: 115K × 100KB = 11.5 GB/sec
Edge Locations: 200
Cache per Edge: 10 TB
```

High-Level Design



Edge Server Components



Key Components

- **PoP**: 200+ locations globally
- **Cache**: LRU/LFU eviction
- **Routing**: Anycast + GeoDNS
- **Invalidation**: Push/Pull
- **Origin Shield**: Regional cache

20. Distributed Cache (Redis/Memcached)

Functional Requirements

- Get/Set operations
- Expiration (TTL)
- Eviction policies
- Atomic operations
- Distributed across nodes
- Replication for availability

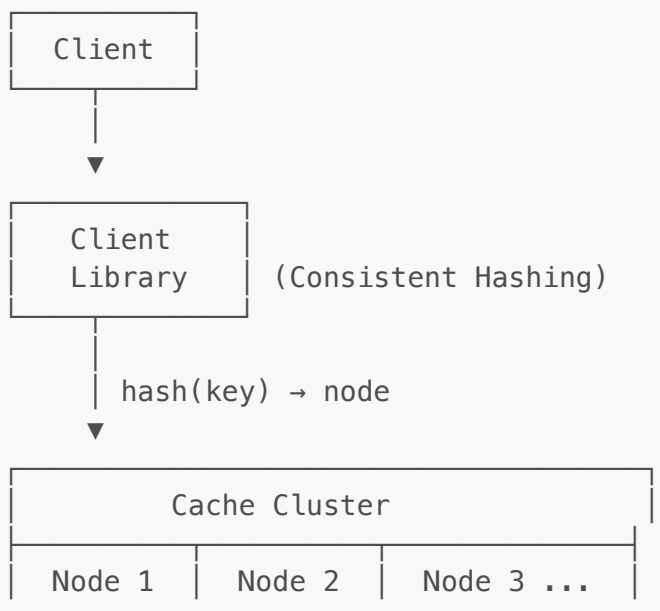
Non-Functional Requirements

- **Low Latency**: <1ms
- **High Throughput**: 100K ops/sec
- **Scalable Horizontally**
- **High Availability**
- **Consistency Options**

Back-of-Envelope Estimation

```
Operations: 1M ops/sec
Avg Key-Value Size: 1KB
Memory per Node: 64 GB
Total Cache: 1 TB
Nodes: 16
Hit Ratio Target: 95%
Replication Factor: 2
```

High-Level Design



```
  | (Master) | (Master) |  (Master)    |
  |          |          |              |
  |    ▼     |    ▼     |     ▼        |
  | Replica  | Replica  |  Replica     |
  └──────────┴──────────┴──────────────┘
```

Eviction Policies

- **LRU** (Least Recently Used)
- **LFU** (Least Frequently Used)
- **FIFO** (First In First Out)
- **Random**
- **TTL-based**

Sharding

- Consistent Hashing
- Hash Slot (Redis Cluster)
- Virtual nodes for balance

Key Components

- **Consistent Hashing**: Even distribution
- **Replication**: Master-slave
- **Persistence**: RDB/AOF snapshots
- **Cluster**: Redis Cluster mode
- **Monitoring**: Hit/miss ratio

---

21. Distributed Task Scheduler

Functional Requirements

- Schedule tasks (one-time/recurring)
- Execute tasks at specified time
- Retry failed tasks
- Task dependencies
- Cron-like scheduling
- Task prioritization

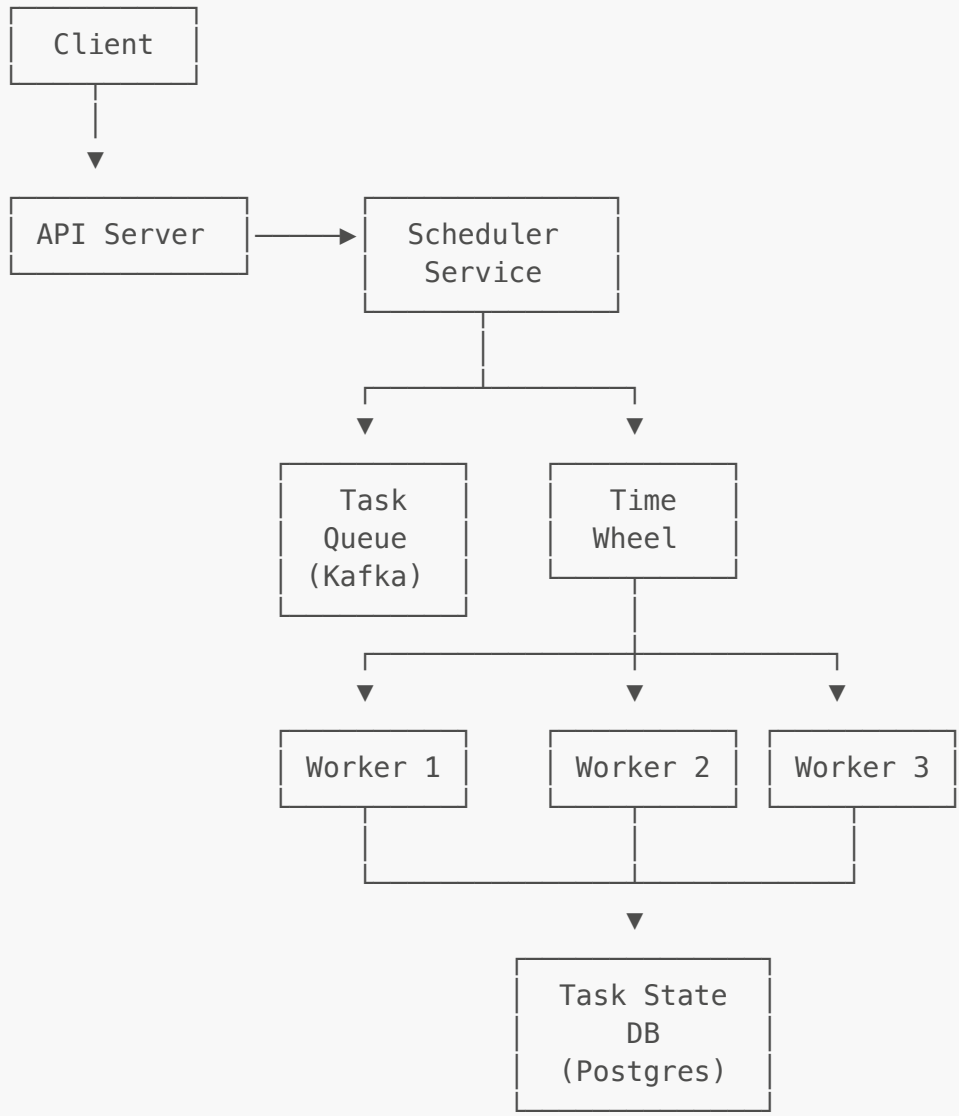
Non-Functional Requirements

- **High Availability**
- **Fault Tolerance**
- **Exactly-Once Execution**
- **Handle**: 1M tasks/day
- **Low Latency** scheduling

Back-of-Envelope Estimation

```
Tasks/day: 10M
Peak: 50M/day = 580/sec
Avg Task Execution: 5 sec
Concurrent Tasks: 3K
Task Metadata: 500 bytes
Storage: 10M × 500 bytes = 5 GB/day
```

High-Level Design



Components

- **Time Wheel**: O(1) scheduling
- **Distributed Lock**: Prevent duplicate execution
- **Task Queue**: Kafka for durability
- **State Machine**: Pending→Running→Success/Failed
- **Retry**: Exponential backoff

Key Features

- Cron expression parsing
- Task dependency DAG
- Dead letter queue
- Monitoring & alerting

---

22. Online Meeting Platform (Teams/Zoom)

Functional Requirements

- Video/audio conferencing
- Screen sharing
- Recording
- Chat
- Participant management
- Breakout rooms

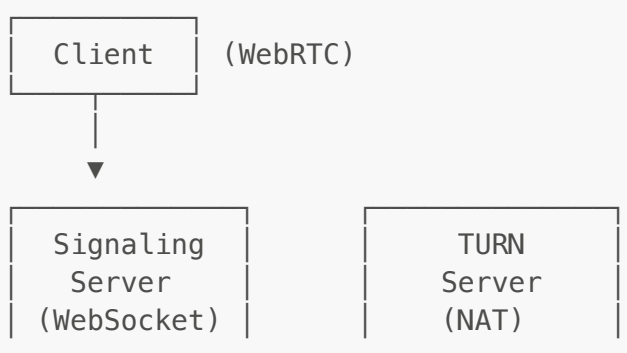
Non-Functional Requirements

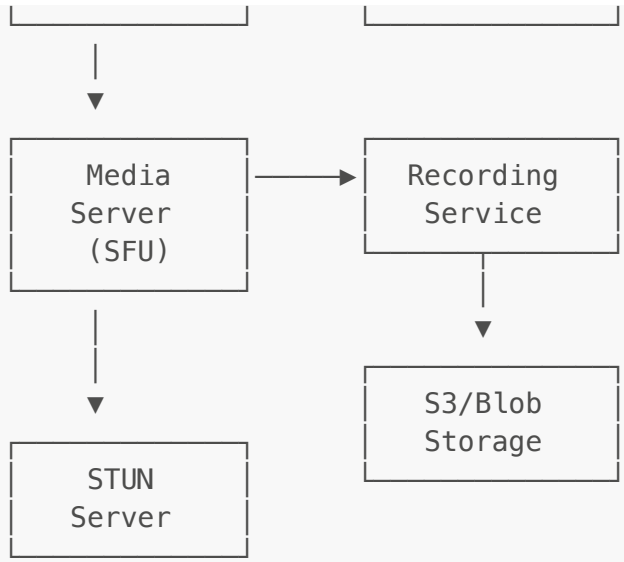
- **Low Latency**: <150ms
- **High Availability**
- **Support**: 1,000 participants
- **HD Video** quality
- **Scalable Infrastructure**

Back-of-Envelope Estimation

```
Concurrent Meetings: 100K
Avg Participants: 10
Total Concurrent Users: 1M
Video Bitrate: 2 Mbps (720p)
Audio Bitrate: 50 Kbps
Bandwidth: 1M × 2 Mbps = 2 Tbps
Recording: 20% meetings × 100KB/sec = 2 TB/hour
```

High-Level Design





Architecture

- **Mesh**: P2P (up to 4-5 participants)
- **SFU**: Selective Forwarding (up to 100)
- **MCU**: Multipoint (large meetings)

Key Components

- **WebRTC**: Browser-based real-time
- **STUN/TURN**: NAT traversal
- **SFU**: Bandwidth optimization
- **Jitsi/Janus**: Open-source SFU
- **Codec**: VP8/VP9/H.264

---

23. Parking Lot System

Functional Requirements

- Track available spots
- Assign parking spot
- Multiple entry/exit gates
- Different vehicle types (car/bike/truck)
- Hourly rate calculation
- Payment processing

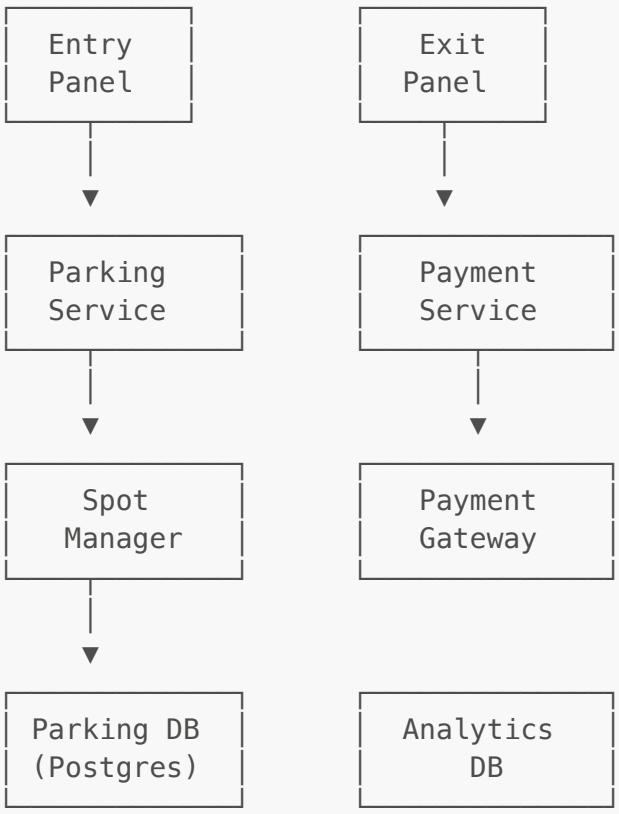
Non-Functional Requirements

- **High Availability**
- **Real-time Updates**
- **Handle**: 10K spots
- **Low Latency**: <100ms
- **Consistent State**

Back-of-Envelope Estimation

```
Parking Spots: 10K
Floors: 10
Entry/Exit Points: 20
Avg Occupancy: 70%
Vehicles/hour: 1K in/out
Peak: 5K/hour
Avg Duration: 2 hours
```

High-Level Design



Data Model

```
ParkingLot {
  floors: [Floor]
}

Floor {
  spots: [Spot]
}

Spot {
  id, type, status
}
```

```
Ticket {
  vehicle, spot, entry_time
}
```

Spot Allocation

- **Strategy**: Nearest to entry
- **Priority**: Reserved > Handicapped > Regular
- **Lock**: Optimistic locking

Key Components

- **Real-time Updates**: WebSocket
- **Payment**: Stripe/PayPal
- **Rate Calculation**: Hourly/daily
- **Database**: ACID transactions

---

24. Metrics Monitoring System

Functional Requirements

- Collect metrics from services
- Store time-series data
- Query and visualize
- Alerting on thresholds
- Aggregation (avg/sum/max/min)
- Retention policies

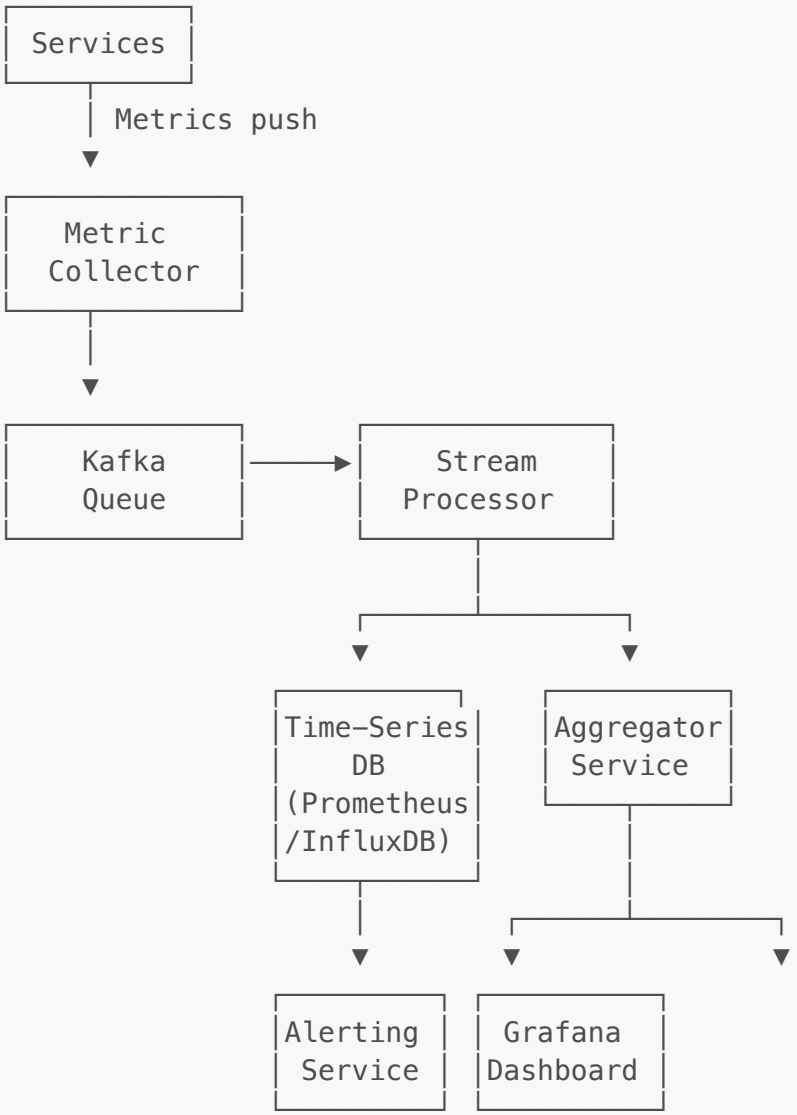
Non-Functional Requirements

- **High Throughput**: 1M metrics/sec
- **Low Query Latency**: <1s
- **Scalable Storage**
- **High Availability**
- **Durability**

Back-of-Envelope Estimation

```
Services: 10K
Metrics per Service: 100
Total Metrics: 1M
Frequency: 10 sec
Data Points/day: 1M × 8,640 = 8.6B
Storage: 8.6B × 20 bytes = 172 GB/day
Retention: 30 days = 5.2 TB
Query Load: 10K queries/sec
```

High-Level Design



Time-Series Optimization

- **Down-sampling**: Reduce resolution over time
- **Compression**: Delta encoding
- **Sharding**: By metric name/time
- **Indexing**: Tag-based queries

Key Components

- **TSDB**: Prometheus/InfluxDB
- **Visualization**: Grafana
- **Alerting**: Alert Manager
- **Storage**: SSTable/LSM Tree

---

25. Proximity Service (Yelp)

Functional Requirements

- Find nearby places
- Search by radius
- Filter by type/rating
- Add/update places
- Reviews and ratings
- Photos

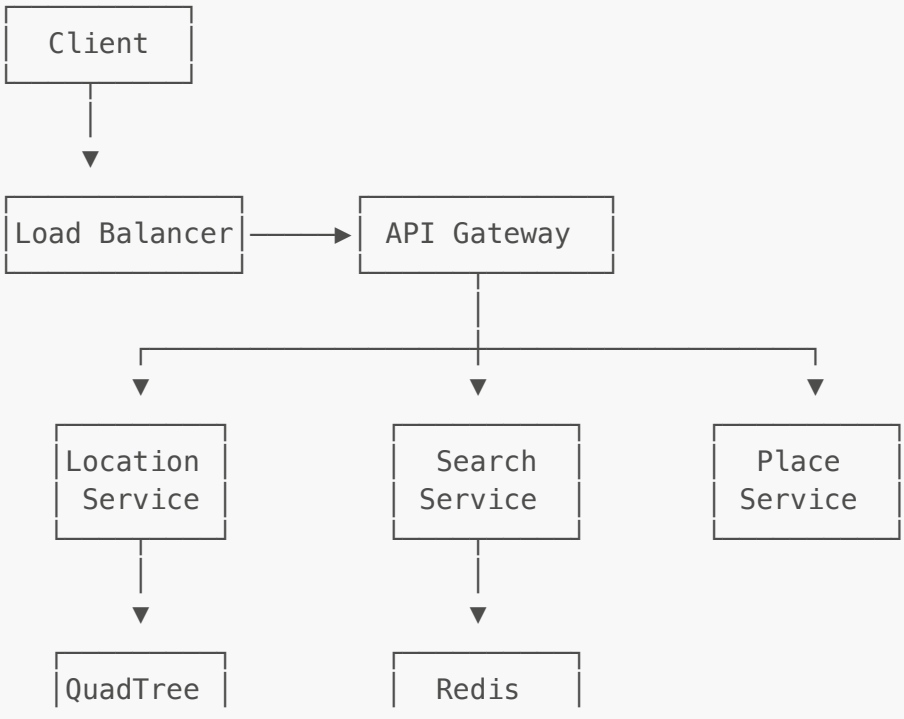
Non-Functional Requirements

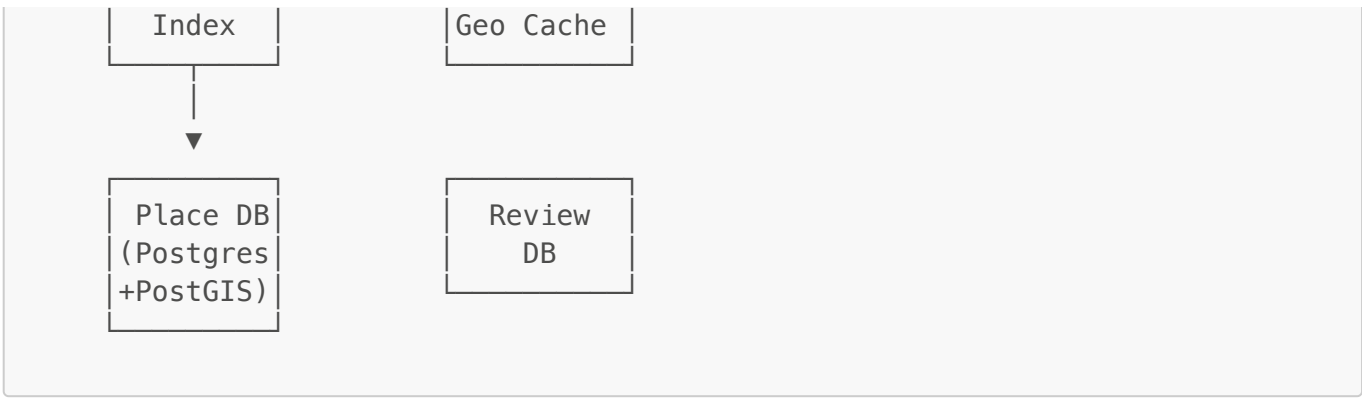
- **Low Latency**: <200ms
- **Handle**: 100M places
- **High Read** throughput
- **Accurate Geospatial** queries
- **Scalable**

Back-of-Envelope Estimation

```
Total Places: 100M
DAU: 50M users
Search Queries: 200M/day = 2.3K/sec
Avg Search Radius: 5 km
Places per Query: 20
Location Updates: 1M/day
Storage: 100M × 2KB = 200 GB
```

High-Level Design





Geospatial Indexing

QuadTree

- Recursively divide space
- Stop at threshold (e.g., 100 places/node)

Geohash

- Encode lat/lng to string
- Prefix matching for proximity

S2 Geometry

- Google's library
- Hierarchical subdivision

Key Components

- **Geohash**: Base32 encoding
- **QuadTree**: In-memory index
- **PostGIS**: PostgreSQL extension
- **Redis Geo**: GEORADIUS command
- **Sharding**: By geographic region

---

26. Distributed Key-Value Store

Functional Requirements

- Put(key, value)
- Get(key)
- Delete(key)
- Distributed across nodes
- Replication
- Consistency guarantees

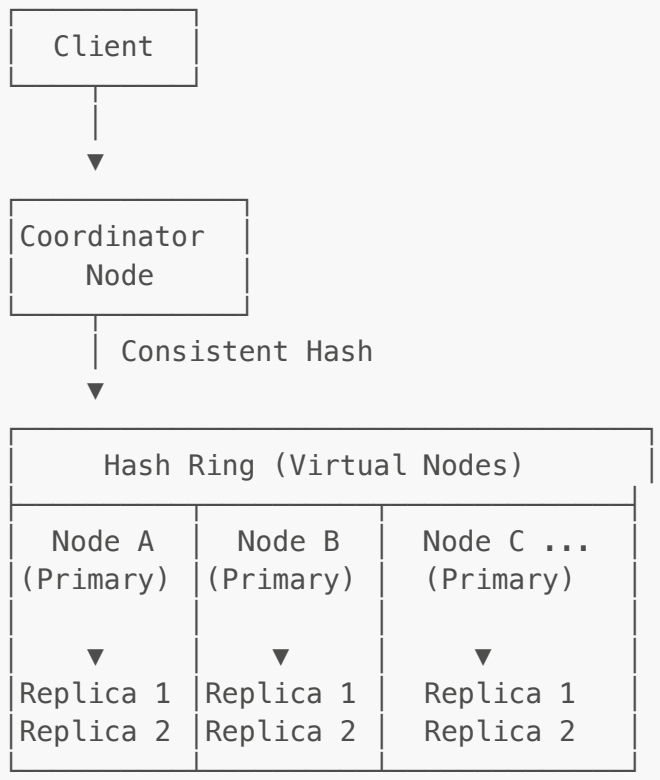
Non-Functional Requirements

- **High Availability**
- **Partition Tolerance**
- **Tunable Consistency**
- **Horizontal Scalability**
- **Handle Node Failures**

Back-of-Envelope Estimation

```
Keys: 1B
Avg Key-Value Size: 1KB
Total Storage: 1 TB
Nodes: 100
Storage per Node: 10 GB
Replication Factor: 3
Total Storage: 3 TB
Operations: 100K/sec
```

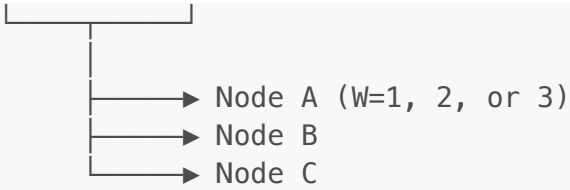
High-Level Design



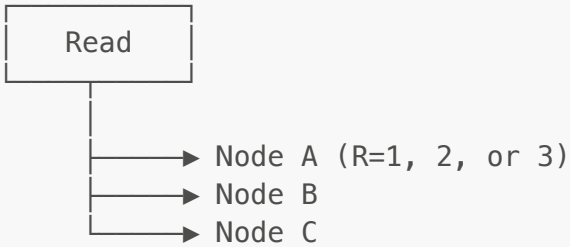
Data Replication

Write





Read



CAP Theorem

- **Consistency**: All nodes see same data
- **Availability**: Every request gets response
- **Partition Tolerance**: Works despite network failures

Quorum

- **W + R > N** (for strong consistency)
- **W=1, R=N** (fast writes, slow reads)
- **W=N, R=1** (slow writes, fast reads)

Key Components

- **Consistent Hashing**: Minimize rebalancing
- **Vector Clocks**: Conflict resolution
- **Gossip Protocol**: Node discovery
- **Merkle Trees**: Data synchronization
- **Examples**: Cassandra, DynamoDB

---

27. Ticket Booking System

Functional Requirements

- Search events/movies
- View seat map
- Select seats
- Book tickets
- Payment processing
- Prevent double booking

- Ticket generation

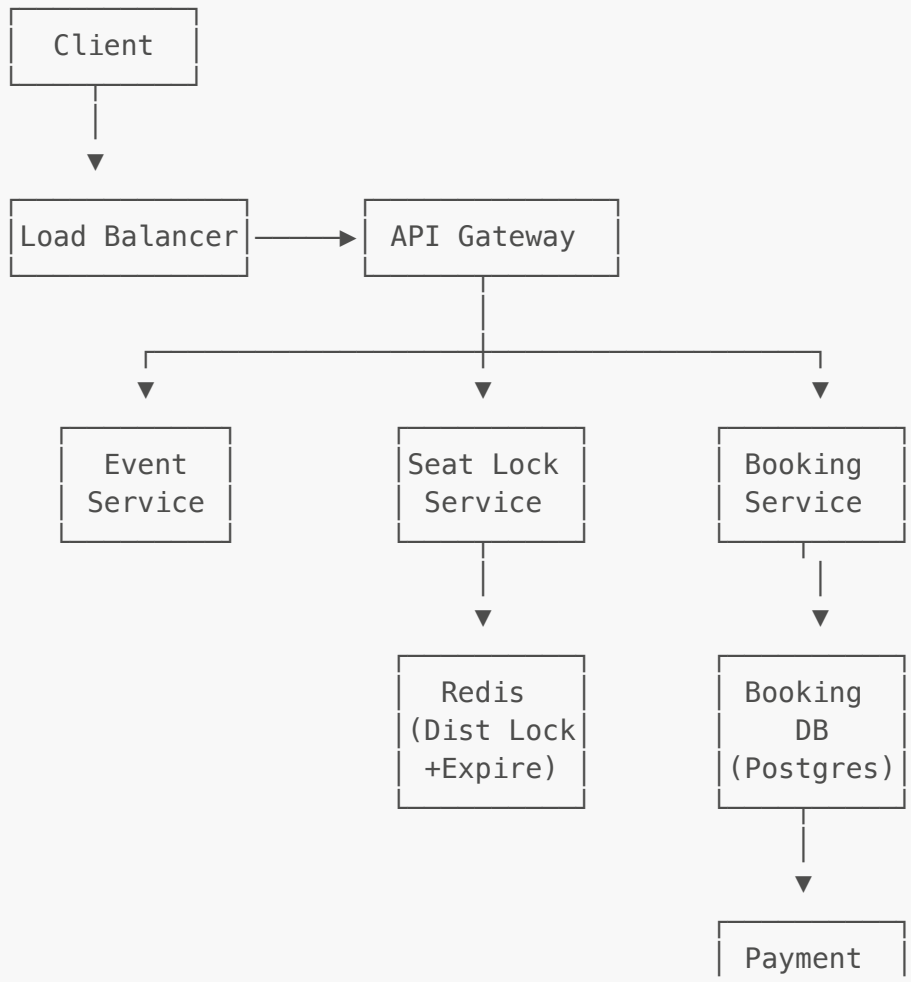
Non-Functional Requirements

- **High Availability**
- **Strong Consistency** (seats)
- **Low Latency**: <500ms
- **Handle**: 10K bookings/sec
- **ACID Transactions**

Back-of-Envelope Estimation

```
Events: 100K active
Seats per Event: 500
Total Seats: 50M
Bookings/day: 10M = 115/sec
Peak: 1,000/sec
Concurrent Users: 100K
Lock Holding Time: 10 min
```

High-Level Design



```
                                    │ Service  │
                                    └──────────┘
```

Concurrency Control

Pessimistic Locking

- Lock seat during selection
- Release after timeout (10 min)
- Prevent double booking

Optimistic Locking

- Version number
- Check before commit
- Retry on conflict

Distributed Lock

- Redis: SETNX with expire
- Redlock algorithm

Booking Flow

1. Select seat
2. Acquire lock (TTL=10min)
3. Show payment page
4. Process payment
5. Confirm booking
6. Release lock
7. Send ticket

Key Components

- **Lock**: Redis distributed lock
- **Transaction**: 2PC or Saga
- **Idempotency**: Prevent duplicate payments
- **Queue**: Handle peak load

---

28. API Gateway

Functional Requirements

- Route requests to services
- Authentication & authorization
- Rate limiting
- Load balancing
- Request/response transformation

- Caching
- Logging & monitoring

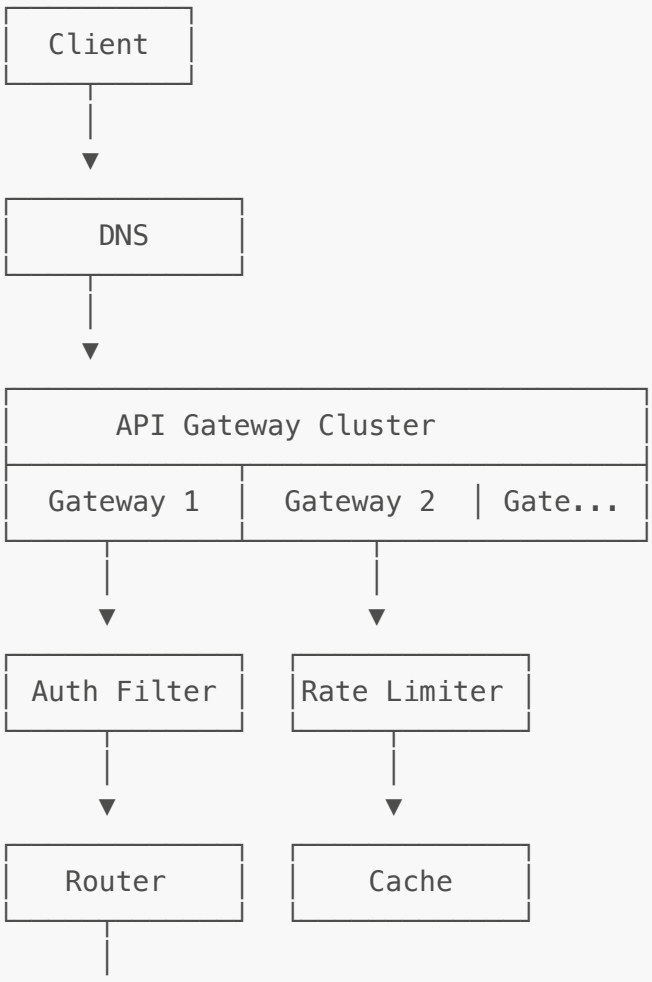
Non-Functional Requirements

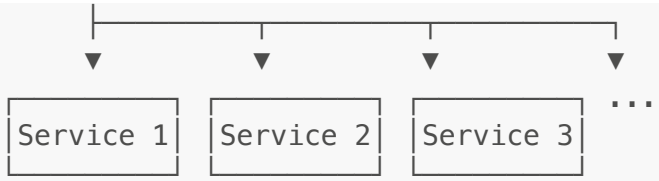
- **High Availability**
- **Low Latency**: <10ms overhead
- **Handle**: 100K requests/sec
- **Scalable Horizontally**
- **Security**

Back-of-Envelope Estimation

```
Services: 100
Requests: 100K/sec
Peak: 500K/sec
Avg Request Size: 10KB
Response Size: 50KB
Bandwidth: 500K × 60KB = 30 GB/sec
Cache Hit Ratio: 40%
```

High-Level Design





Features

Routing

- Path-based
- Header-based
- Service discovery

Authentication

- JWT validation
- OAuth 2.0
- API keys

Rate Limiting

- Per user/IP
- Token bucket

Load Balancing

- Round-robin
- Least connections
- Weighted

Circuit Breaker

- Prevent cascade failures
- Fallback responses

Transformation

- Request/response mapping
- Protocol conversion

Key Components

- **Implementation**: Kong/Nginx/Envoy
- **Service Registry**: Consul/etcd
- **Cache**: Redis
- **Monitoring**: Prometheus
- **Logging**: ELK stack

29. News Feed System (LinkedIn/Facebook)

Functional Requirements

- Generate personalized feed
- Create posts (text/images/videos)
- Like, comment, share
- Follow users/pages
- Ranking algorithm
- Real-time updates

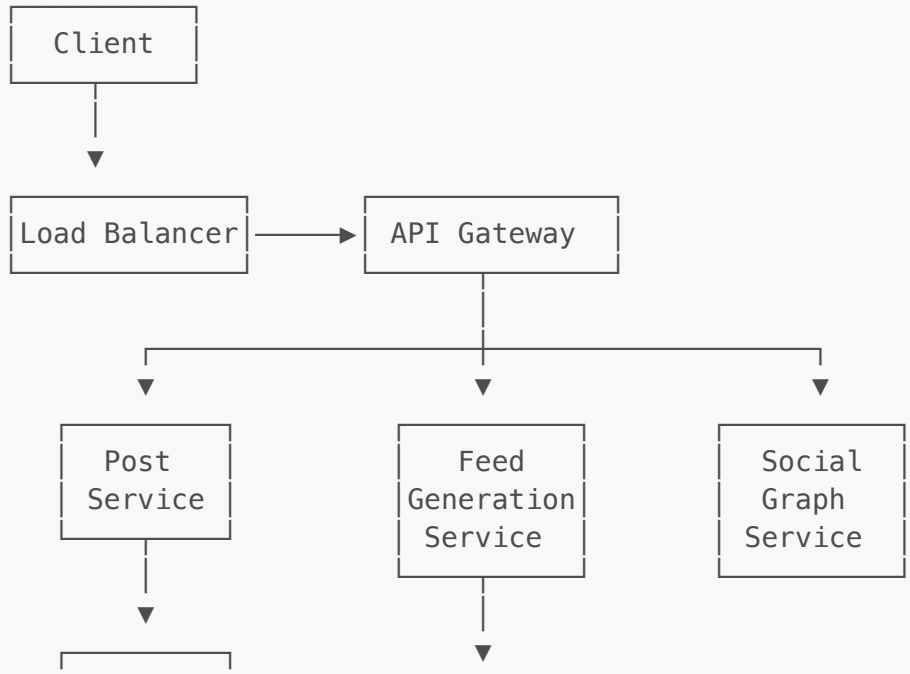
Non-Functional Requirements

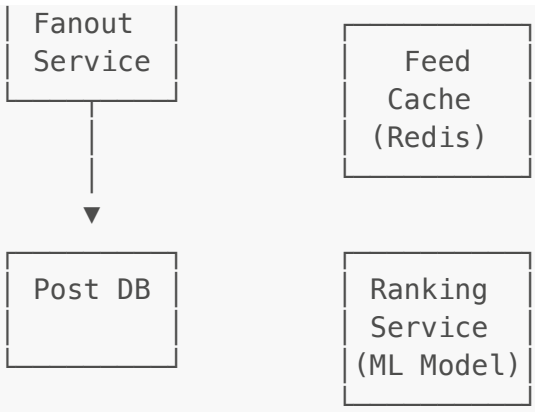
- **Low Latency**: <200ms
- **High Availability**
- **Handle**: 1B users
- **Eventually Consistent**
- **Scalable**

Back-of-Envelope Estimation

```
DAU: 500M users
Avg Connections: 200
Posts: 100M/day = 1,150/sec
Feed Reads: 500M × 5 = 2.5B/day = 29K/sec
Fan-out: 1,150 × 200 = 230K writes/sec
Storage: 100M × 1KB = 100 GB/day
```

High-Level Design





Fanout Strategies

Fan-out on Write (Push)

- Precompute feeds
- Fast reads
- Slow writes for celebrities

Fan-out on Read (Pull)

- Compute on request
- Slow reads
- Fast writes

Hybrid

- Push for active users
- Pull for celebrities
- Best of both

Ranking Algorithm

- Recency score
- Engagement score (likes/comments)
- User affinity
- Content type preference
- ML model (e.g., EdgeRank)

Key Components

- **Fan-out**: Kafka for async processing
- **Cache**: Redis for feeds
- **Ranking**: Personalization ML
- **Graph**: Social connections
- **Database**: Cassandra for posts

---

30. Search Engine (Google)

Functional Requirements

- Crawl web pages
- Index content
- Search and rank results
- Handle typos/synonyms
- Autocomplete
- Featured snippets

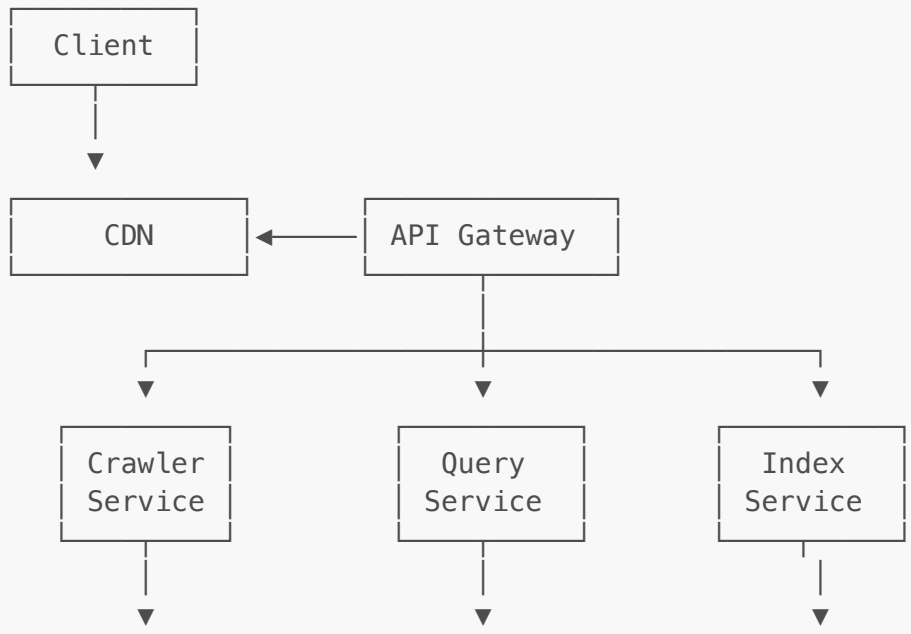
Non-Functional Requirements

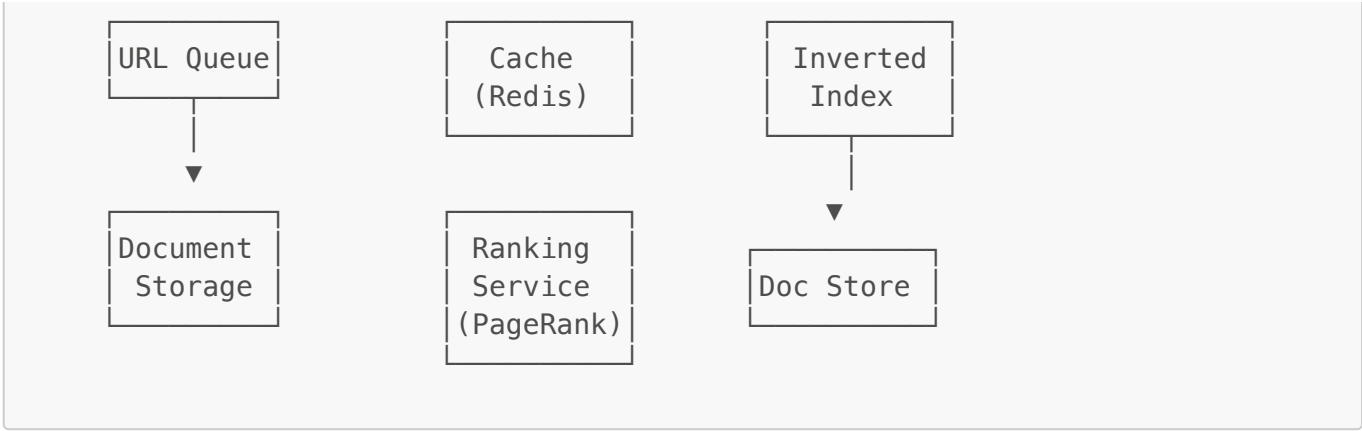
- **Low Latency**: <100ms
- **High Availability**
- **Handle**: Billions of pages
- **Accurate Ranking**
- **Scalable**

Back-of-Envelope Estimation

```
Web Pages: 100B
Index Size: 100B × 10KB = 1 PB
Queries: 100B/day = 1.15M QPS
Crawl: 100B pages/month
Avg Query: 3 words
Results: Top 10 per query
Cache Hit: 40%
```

High-Level Design





Inverted Index

```
Word → [Doc1, Doc2, Doc3...]

Example:
"hello" → [doc1, doc5, doc7]
"world" → [doc1, doc3]

Query: "hello world"
Intersection: [doc1]
```

Ranking Factors

PageRank

- Link analysis
- Importance score

TF-IDF

- Term frequency
- Inverse document frequency

User Signals

- Click-through rate
- Dwell time

Freshness

- Recent content

Domain Authority

Key Components

- **Crawler**: Distributed, polite

- **Indexer**: MapReduce for building index
- **Ranker**: ML models
- **Sharding**: By term/document
- **Cache**: Hot queries in Redis

Conclusion

This document covers 30 essential High-Level Design questions commonly asked in Microsoft interviews. Each design includes:

- **Functional Requirements**: What the system should do
- **Non-Functional Requirements**: Quality attributes (availability, latency, scalability)
- **Back-of-Envelope Estimations**: Traffic, storage, and bandwidth calculations
- **High-Level Design Diagrams**: Architecture visualization
- **Key Components**: Critical technologies and algorithms

Preparation Tips

1. **Master the Fundamentals**: Understand CAP theorem, consistency models, sharding, replication
2. **Practice Estimations**: Get comfortable with back-of-envelope calculations
3. **Know Trade-offs**: Be ready to discuss pros/cons of different approaches
4. **Real-World Systems**: Study how actual systems (AWS, Google, Meta) solve these problems
5. **Communication**: Practice explaining your designs clearly and concisely

Common Patterns

- **Caching**: Redis, CDN
- **Message Queues**: Kafka, RabbitMQ
- **Databases**: SQL vs NoSQL, sharding strategies
- **Load Balancing**: Round-robin, least connections, consistent hashing
- **Monitoring**: Metrics, logging, alerting

Good luck with your Microsoft interview! 🚀

Note: This document is a quick reference guide. For deeper understanding, study each system in detail and explore real-world implementations.